The Australian National University

School of Computing

JHQ-GPU

Representation-Aware GPU ANN Search and Workload-Level Refinement Calibration

Rui Luo

Supervisor: Dr Mengxuan Zhang

A research thesis submitted for

COMP8800 — Advanced Computing Research Project

October 2026

Abstract

Reducing approximate nearest-neighbour (ANN) query time requires efficient screening and selective refinement. This thesis presents JHQ-GPU, which implements JHQ through representation-aware execution, GPU index construction and workload-level budget calibration. Cartesian codebook factorisation replaces each 256-entry distance table with two 16-entry tables, preserving the real-arithmetic represented distance while reducing table storage eightfold. Candidate-parallel scanning, packed access and lane-cooperative refinement complement this design. Refinement budgets are calibrated through sampled top-k agreement with a larger reference budget, without ground-truth labels or a learned predictor.

Experiments on an RTX 5090 cover six datasets. Two-group tables lead in 22/24 configurations; the largest index searches 17.78 million vectors at 20.44 GiB resident memory. Full-pool calibration meets a 10^{-3} held-out recall-loss target relative to the best tested budget in 12/12 cells, with maximum loss 0.0006; smaller samples expose workload-dependent quality risk. At recall 0.97 and batch 1,000, JHQ leads in three of four datasets and is near parity with IVF-RaBitQ on arXiv. OpenAI runs use TF32; FP32 checks retain favourable comparisons under a TF32-trained cache. At batch 8 and the same recall, CAGRA fp32 leads on all four datasets. These results locate the execution, quality and capacity trade-offs of GPU JHQ.

Acknowledgements

I thank Dr Mengxuan Zhang for supervising this project and acknowledge the JHQ authors and open-source ANN/GPU software cited throughout this thesis.

Research and AI assistance. This thesis extends joint JHQ-GPU work with Mengxuan Zhang [21] and builds on JHQ [16]. OpenAI Codex assisted with substantive drafting, revision, LaTeX, analysis and code/log checks [30]; ChatGPT supplied frameworks and reviews [29]; Anthropic Claude assisted with implementation, experiments and review [4]. Selected prompts and usage details are available in the AI-use record. The author is responsible for the submitted claims.

Contents

Abstract i

Acknowledgements i

Notation and Abbreviations vi

1 Introduction 1
- 1.1 Context and Practical Objectives . . . 1
- 1.2 Problem Definition and Scope . . . 1
- 1.3 Three Linked Technical Challenges . . . 1
- 1.4 Research Questions and Falsifiable Claims . . . 2
- 1.5 Research Contributions . . . 2

2 Background and Related Work 4
- 2.1 ANN Objective and Metrics . . . 4
- 2.2 IVF, PQ, ADC and Refinement . . . 4
- 2.3 JHQ Representation and Assumptions . . . 5
- 2.4 GPU Execution Concepts . . . 6
- 2.5 Graph Search on GPUs . . . 6
- 2.6 Quantised Search and Fast Tables . . . 6
- 2.7 Research Gap and Design Choices . . . 7

3 Research Design and Measurement 8
- 3.1 Research Questions and Comparisons . . . 8
- 3.2 Datasets, Queries, Ground Truth and Splits . . . 8
- 3.3 Timing, Variance and Memory Accounting . . . 9
- 3.4 Baselines and Frontier Construction . . . 10
- 3.5 Experimental Records and Reproducibility . . . 11
- 3.6 Evidence Strength and Threat Controls . . . 11

4 GPU Index Design and Construction 12
- 4.1 End-to-End Build and Search Boundaries . . . 12
- 4.2 Training, Rotation and Codebook Generation . . . 13
- 4.3 IVF Assignment, IDs and Physical Memory Layout . . . 14
- 4.4 Resident Storage, Workspace and Index Lifecycle . . . 16
- 4.5 Construction Cost and Tested Scale . . . 17
- 4.6 Index Representation and Safety Contracts . . . 18

5 Representation-Aware GPU Search 20
- 5.1 Query Path and Four Distinct Ranking Objects . . . 20
- 5.2 Exact Cartesian LUT Factorisation . . . 21
- 5.3 Packed Codes and Candidate-Parallel Memory Access . . . 23
- 5.4 Exact Global Primary Top-c via Multi-Block Scanning . . . 23
- 5.5 Shared Complete-Key Thresholds and Execution Scheduling . . . 26
- 5.6 Lane-Cooperative Residual Scoring and Final Top-k . . . 29
- 5.7 IVF Probe Selection as an Independent Primitive . . . 30

6 Workload-Level Budget Calibration 31
- 6.1 Why Fixed Refinement Budgets Can Be Wasteful . . . 31
- 6.2 GT-Free Sampled Output Agreement . . . 32
- 6.3 Budget Search, Nestedness and Tie Safety . . . 33
- 6.4 Budget Baselines and Their Questions . . . 35
- 6.5 Hold-out Risk, Calibration Cost and Payback . . . 35

6.6 Policy Failure Modes and Applicability 36

7 Experimental Evaluation **37**
7.1 Experimental Protocol 37
7.2 Modern GPU ANN Recall–QPS Frontiers 37
7.3 Cartesian LUT, Packed Loads and Lane Refinement 40
7.4 Global Top-c, Shared Bound and Probe Correctness/Ablation 42
7.5 Held-out Budget Quality, Throughput and Payback 43
7.6 Batch Regimes, G Scheduling and Workload Skew 47
7.7 Cold Construction, Resident Memory and Tested Scale 48
7.8 Numerical Robustness, Anomalies and Recall Attribution 50

8 Discussion and Future Work **52**
8.1 Three Cross-Layer Insights 52
8.2 Two Deep Negative-Result Case Studies 52
8.3 Threats to Internal and External Validity 53
8.4 Testable Research Directions 54
8.5 Conditional Deployment Recommendation 54

9 Conclusion **55**
9.1 Answers to the Four Research Questions 55
9.2 Closing Principle and Outlook 55

Bibliography **56**

A Reproducibility and Experiment Manifest **59**
A.1 Source, Environment and Trained States 59
A.2 Measurement Contracts 59
A.3 Budget Harness Contract 59
A.4 Supplementary Evidence 59
A.5 Strict Selection, Small Batches and Artifact Access 60

B Supplementary Results and Correctness Checks **61**
B.1 Parameter Coverage 61
B.2 Correctness and Numerical Evidence 62
B.3 Secondary Results and Evidence Status 62
B.4 Construction Capacities and Precision Anchors 62
B.5 Artifact Access 63

List of Figures

1.1 Problem, mechanism and evidence . . . 2
2.1 Candidate and precision cascade . . . 5
3.1 Host and device timing boundaries . . . 10
4.1 Build and query architecture . . . 12
4.2 List-ordered physical index layout . . . 15
5.1 Query stages and selection boundaries . . . 20
5.2 Packed candidate-parallel access . . . 23
5.3 Local coverage and truncated pairwise merging . . . 25
5.4 Shared complete-key filtering . . . 27
5.5 Sixteen-lane residual refinement . . . 29
6.1 Work affected by the refinement budget . . . 31
6.2 Workload-level calibration and evaluation . . . 32
6.3 Nested and non-nested candidate pools . . . 34
7.1 E1 recall–throughput frontiers . . . 38
7.2 TF32 and FP32 precision frontiers . . . 39
7.3 Small-batch competitive regions . . . 40
7.4 Cartesian lookup-table grouping . . . 41
7.5 Packed access and refinement controls . . . 42
7.6 Held-out risk across sample sizes . . . 45
7.7 Finite-pool risk prediction and observation . . . 46
7.8 Direct cumulative calibrated-reuse time . . . 47
7.9 Batch-dependent execution regimes . . . 48
7.10 Recorded construction costs . . . 49
7.11 Resident-memory decomposition . . . 50
7.12 Large-dataset search frontiers . . . 51
8.1 Mailman and rank-oriented coding . . . 53

List of Tables

1.1 Technical contributions and evidence boundaries . . . 3
2.1 Design alternatives and research gap . . . 7
3.1 Research questions and experimental controls . . . 8
3.2 Dataset and training configuration . . . 9
3.3 Principal E1 tuning dimensions . . . 11
4.1 Per-vector storage model . . . 16
4.2 Analytical row-array occupancy by construction phase . . . 17
4.3 Original-input FP64 recoding audit . . . 19
4.4 Representation contracts and audit coverage . . . 19
5.1 Query-stage selection contracts . . . 21
5.2 Cartesian lookup-table trade-offs . . . 22
6.1 Budget controls and their roles . . . 35
7.1 Cross-system comparison at recall 0.97 . . . 37
7.2 FP32 sensitivity at matched configurations . . . 39
7.3 Small-batch matched-recall ratios . . . 40
7.4 Refinement gains at 128 probes . . . 41
7.5 Strict computed-score selection checks . . . 42
7.6 Probe-selection throughput ratios . . . 43
7.7 Full-pool budget calibration . . . 44
7.8 Sampled-budget held-out quality . . . 44
7.9 Sampled-calibration economics . . . 45
7.10 Direct costs for 500 reuse batches . . . 46
7.11 Integrated execution and scheduling gains . . . 47
7.12 Cold construction and resident memory . . . 49
7.13 E1 allocation ranges . . . 50
8.1 Threats to validity . . . 54
A.1 Component and run provenance . . . 59
B.1 Exact E1 configuration counts . . . 61
B.2 Validation coverage . . . 62
B.3 Independent construction-buffer capacities . . . 62
B.4 Same-session precision anchors . . . 63

Notation and Abbreviations

Symbol	Meaning
N, d	Number of database vectors and original dimension
M, D_s	Number of primary subspaces and dimensions per subspace
$a_{\text{add}}, a_{\text{in}}, a_{\text{assign}}$	Configured add, staged input, and independently allocated assignment rows
n_b, S_{stage}	Pipeline buffer count and byte budget per input/rotated array
Π	Square orthogonal transform; $q' = \Pi q$
B, B_r	Primary bits per subspace and residual bits per coordinate
$\rho(p), o_\ell$	Input ID at physical position p, and start of IVF list ℓ
k	Number of returned neighbours
n_c, c	Routed candidates and primary survivors receiving refinement
α	Refinement multiplier, with $c = \max\{k, \lceil \alpha k \rceil\}$
S, ϵ	Calibration sample size and allowed disagreeing result slots
$C_\alpha, \mathcal{R}_\alpha$	Valid primary survivor set and returned ID set at budget α
$\alpha_{\text{ref}}, B_{\text{reuse}}$	Reference refinement budget and number of subsequent reuse batches
g_{LUT}	Number of groups in a factorised lookup table
g_{scan}	Number of scan groups per query; zero labels the one-block path
$C_{\text{buf}}, T_{\text{blk}}$	Scan buffer capacity and threads per block
$n_{\text{LUT}}(g_{\text{LUT}})$	Number of LUT entries per primary subspace
$\kappa(D, p)$	Packed complete-key encoding for bound publication
ℓ_{ref}	Number of cooperating lanes per refined candidate
D_{raw}	Squared distance between original vectors
D_{P}	Distance under the primary reconstruction
D_{H}	Refined ranking score: reconstructed squared distance plus a query-only constant

Abbreviation	Meaning
ANN	Approximate nearest neighbour
ADC	Asymmetric distance computation (distinct from the ADC conference)
IVF	Inverted file
JHQ	Johnson–Lindenstrauss enhanced hierarchical quantisation
LUT	Lookup table
PQ	Product quantisation
QPS	Queries per second
GT	Ground truth
FP32	IEEE single-precision storage and arithmetic, with multiplication mode stated separately
TF32	Tensor-core multiplication format with reduced input precision and FP32 accumulation
CV	Coefficient of variation of candidate counts

1 Introduction

1.1 Context and Practical Objectives

ANN avoids scoring every full-precision vector. Graph search uses navigation; compressed IVF scans selected lists with cheap scores [18, 28, 33]. Here the data reach 17.78 million vectors and 3,072 dimensions. Stella's FP32 vectors alone need 67.81 GiB (section 7.7). Compact codes save space; their search cost still depends on tables, memory access and selection.

JHQ screens with primary codes, then refines a shortlist using residuals [16]. This leaves two questions: how to run the scan and how many candidates to refine. Reusing the trained index lets us vary each choice. We measure recall, host-visible speed, resident memory and construction cost, including cases where JHQ loses.

1.2 Problem Definition and Scope

The target is raw squared-L2 top-k for query q over $\mathcal{X} \subset \mathbb{R}^d$. IVF routes the query; primary score D_{P} selects up to $c = \max\{k, \lceil \alpha k \rceil\}$ survivors; residual score D_{H} selects the final results. These reconstructed scores differ from raw distance. The refined value additionally contains a query-only constant, as defined in section 2.3. Selection is exact for its computed score and tie rule; GT measures ANN quality.

We retain JHQ's square rotation, Cartesian primary codes and primary-relative residuals. The work covers GPU construction, layout, execution and one workload-level refinement multiplier. Unlabelled output agreement selects the multiplier for reuse on a stable workload. It supplies neither a per-query predictor nor an absolute-recall guarantee.

E1 uses one RTX 5090, squared-L2 top-10 and batch 1,000 on four datasets. Component and budget studies use six datasets; T5 adds batches 8/32 (section 7.2.5). The compressed index stays on the GPU, with host preparation and query transfers. Updates, multi-GPU search, host-backed residuals and billion-scale deployment are untested.

1.3 Three Linked Technical Challenges

Figure 1.1 connects three challenges to the mechanisms and evidence used to address them.

First, the primary codebook has useful mathematical structure that a direct GPU port can overlook. Each eight-bit subspace code indexes 256 distances. Its Cartesian coordinates permit an exact 16+16 factorisation, but more aggressive splitting adds lookups and arithmetic. The challenge is to exploit separability without assuming that the smallest table gives the fastest scan. Candidate-parallel word access and residual lane cooperation also need different data mappings.

Second, all global winners might lie in one part of a query's routed pool. Dividing the required survivor count between groups can therefore lose answers. Local retention and merging preserve coverage; shared bounds reduce extra work. These operations add overhead, so batch size and skew must guide the choice. IVF-list selection remains separate from database-candidate retention.

Third, a large survivor budget wastes refinement, while a small one can discard neighbours. Reference agreement needs no GT, but inherits reference errors. Calibration also costs time. We test quality and total cost over subsequent batches.

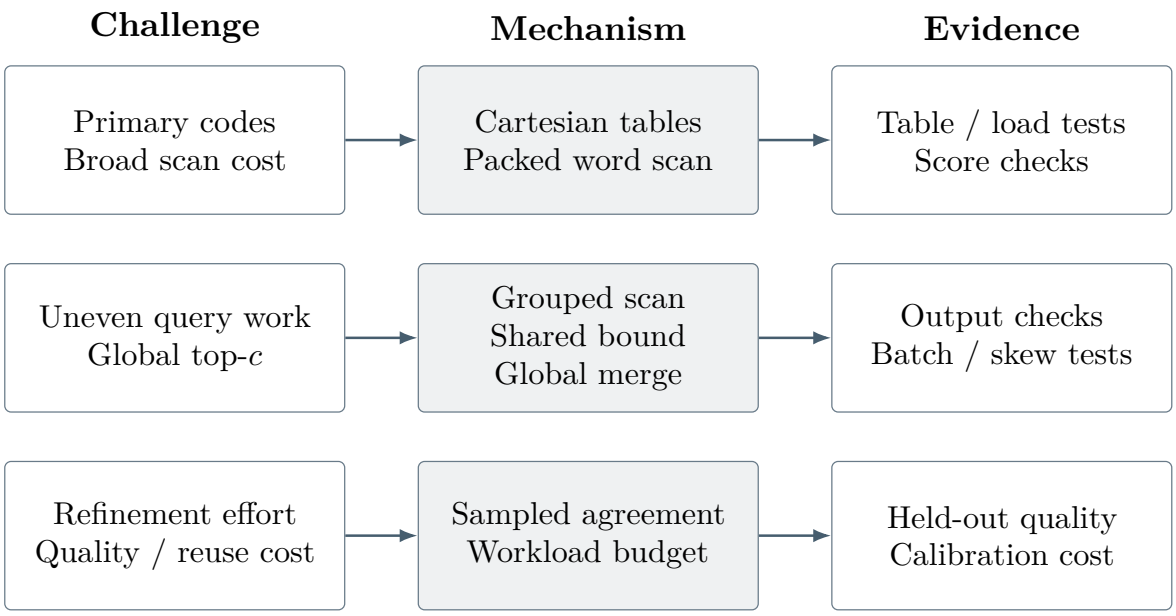


Figure 1.1. Problem, mechanism and evidence. The arrows organise research questions; they do not imply that independently measured speedups multiply or that an unlabelled agreement rule guarantees GT recall.

1.4 Research Questions and Falsifiable Claims

RQ1: Construction and capacity. How can JHQ become a GPU-resident searchable index, and what construction cost and capacity are demonstrated (section 7.7)? Resident size and construction peak are measured separately.

RQ2: Correct and efficient execution. Which scoring, selection and scheduling choices improve speed while preserving the specified scores and tie rules? Paired experiments and reference checks answer this in sections 7.3, 7.4 and 7.6.

RQ3: Budget quality and total cost. Can unlabelled agreement choose a useful survivor budget and repay calibration? Held-out tests compare fixed budgets, sampled/FULL rules and enumeration (section 7.5). The GT-labelled oracle is an evaluation control.

RQ4: Competitive operating regions. Where does JHQ compete with modern GPU ANN methods at matched recall and a common host-to-host boundary? The answer uses independently tuned E1 frontiers and explicit memory definitions (section 7.2). Int8 graph competitors, high-recall reversals and absent coverage are part of the answer.

Chapter 3 turns these questions into controlled comparisons. Together they separate what was built, whether its execution is correct, what effort a workload requires, and whether the resulting system has useful operating regions.

1.5 Research Contributions

Table 1.1 states three linked project contributions and their inherited foundations. The core contribution is the adaptation and evaluation of a particular representation–execution–budget combination. JHQ, Cartesian quantisation, IVF, bitonic/radix selection, merging and CUDA Graph execution retain their prior-art status.

Table 1.1. Technical contributions and their evidence boundaries.

	Inherited foundation	Project contribution	Evidence
C1	JHQ rotation, codebooks and hierarchy	GPU training/encoding, bounded batch construction and list-ordered packed representation	Search of 17.78M vectors at 20.44 GiB resident; repeated train+add 10.54–10.65 s; sampled construction peak 22.58 GiB
C2	Cartesian additivity and established GPU primitives	Integrated factorised scoring, complete-key grouped retention/bounds, lane refinement and execution scheduling	Two-group LUTs lead in 22/24 cells; scheduled execution 2.727× at batch 8; explicit correctness scope
C3	Candidate expansion and output-agreement ideas	Workload-level budget rule, conditional selection analysis, held-out controls and cost/risk evaluation	FULL meets the relative-loss target in 12/12 cells; smaller samples expose risk; four cells directly measure calibrated reuse

Performance depends on batch size and quality target; Chapters 7–8 report the unfavourable regions and numerical limits.

2 Background and Related Work

This chapter covers candidate access, quantisation and selection, then places JHQ's execution and budget choices against existing GPU ANN methods.

2.1 ANN Objective and Metrics

For database $\mathcal{X} = \{x_i\}_{i=0}^{N-1} \subset \mathbb{R}^d$ with stable IDs, exact top-k selects the smallest distances

$$D_{\text{raw}}(q, x) = \|q - x\|_2^2. \tag{2.1}$$

We use the stored GT top-10 set, including its tie choices. Recall@10 measures how many of those IDs ANN recovers (equation (3.1)); it is not a worst-case distance guarantee.

We measure query throughput, batch-call latency, construction cost and storage. These are separate costs: faster batch processing does not establish concurrent-request tail latency, and resident index bytes exclude some workspace and temporary allocations. Chapter 3 gives the boundaries.

Tuning gives a finite recall–throughput frontier. A measured threshold chooses a qualifying configuration; interpolation estimates between points. Missing coverage proves neither a win nor an impossibility. Section 3.4 fixes both estimators and endpoint tolerance.

2.2 IVF, PQ, ADC and Refinement

IVF partitions the database around coarse centroids and stores a list of vectors assigned to each centroid. A query chooses n_{probe} lists, producing a routed universe $U(q)$ of n_c candidates. Visiting fewer lists reduces access work but can exclude a true neighbour before any fine scoring. The centroids are a routing device; their distance ordering is a different selection problem from choosing database vectors.

Product quantisation divides a vector into M coordinate subspaces and represents each with a label into a small learned codebook [17]. If $\widehat{x}_m = \mathcal{C}_m[z_m]$, asymmetric distance computation (ADC) keeps the query unquantised and scores

$$\|q - \widehat{x}\|^2 = \sum_{m=0}^{M-1} \|q_m - \mathcal{C}_m[z_m]\|^2. \tag{2.2}$$

A query-specific table precomputes each subspace distance. Scanning then uses code loads, table lookups and accumulation. The product is between subspaces; the centroids *within* a learned subcodebook need not have a further coordinate product. This distinction determines whether Chapter 5's exact internal table factorisation is valid.

IVF-PQ often encodes $x - \mu_{\ell(x)}$. JHQ instead uses a residual relative to the primary reconstruction after rotation. Adding the IVF centroid during JHQ reconstruction would therefore add a term the encoder never removed. Candidate expansion and more accurate refinement are established ideas [10, 18].

With at least k valid routed candidates, the survivor cap is $c(\alpha) = \max\{k, \lceil \alpha k \rceil\}$. The output and survivor sets satisfy

$$\mathcal{R}_\alpha(q) \subseteq C_\alpha(q) \subseteq U(q) \subseteq \mathcal{X}. \tag{2.3}$$

Refinement can reorder retained vectors, but cannot recover excluded IDs. Stage-wise GT coverage bounds final recall; measuring each stage's loss needs counterfactual scoring over a fixed universe.

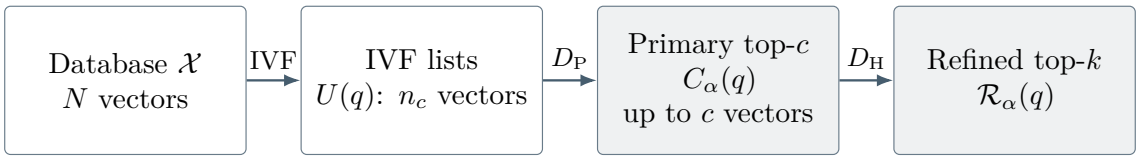


Figure 2.1. Candidate and precision cascade. IVF selects lists; primary scoring selects vectors; only survivors read residuals. The hierarchy is inherited. This diagram defines logical sets and score fidelity, not measured recall losses or GPU kernel boundaries.

Broad scoring takes $O(n_c M)$ lookups; refinement takes $O(cd)$ coordinate work. Actual latency also includes access and selection. Probes change the routed universe U; α changes retention within it. They control different losses.

2.3 JHQ Representation and Assumptions

2.3.1 What is inherited

JHQ combines an orthogonal transformation, an analytical Cartesian primary code and a higher-precision residual layer [16]. Here the transform is square: $\Pi \in \mathbb{R}^{d\times d}$, $\Pi^\top \Pi = I$, $y = \Pi x$ and $q' = \Pi q$. Therefore

$$\|q - x\|^2 = \|\Pi(q - x)\|^2. \tag{2.4}$$

Although the prior work calls this an orthogonal JL transformation, this configuration does not reduce dimension. It preserves real-arithmetic distance exactly; quantisation subsequently introduces approximation. Actual FP32 orthogonality and GEMM error are separate implementation checks.

Partition y into M subspaces of dimension $D_s = d/M$. With B bits per primary subspace, $K = 2^B$ labels are formed from $L = 2^{B/D_s}$ scalar levels, requiring the implemented Cartesian parameters to make L an integer. Writing Φ for the standard normal CDF, the constructor uses

$$a_i = \sigma\Phi^{-1}\left(\frac{i - 1/2}{L}\right), \quad i = 1, \ldots, L, \qquad \mathcal{C}_m = \{a_1, \ldots, a_L\}^{D_s}. \tag{2.5}$$

At the evaluated $B = D_s = 8$, $L = 2$ and $\mathcal{C}_m = \{-a, +a\}^8$, with $a \simeq 0.67449\sigma$. Each byte records eight coordinate signs. The implementation estimates σ from vector energy; unit-normalised inputs motivate the scale $1/\sqrt{d}$. This representation is prior art, not an invention of the GPU execution layer.

Residuals are $r = y - \widehat{p}$, giving reconstruction $\widehat{y} = \widehat{p} + \widehat{r}$. The evaluated codes use $d/8$ primary bytes and d residual bytes per vector, before metadata. Refining fewer rows reduces reads, not stored residual size.

2.3.2 Model assumptions and score semantics

Random rotation can spread coordinate energy, motivating a shared Gaussian-inspired scalar model. It does not force an arbitrary realised database to have independent, zero-mean Gaussian coordinates. For a fixed rotation and empirical database mean μ, the transformed mean is $\Pi\mu$ and $\|\Pi\mu\| = \|\mu\|$. Averaging over random rotations is different from centring one fixed corpus. Even a uniformly random point on a sphere has dependent coordinates through its fixed norm. Consequently, isometry is an algebraic guarantee; the distributional approximation is a modelling assumption. Chapter 8's centred-coding pilot examines one practical consequence of that distinction.

The inherited primary levels are Gaussian mid-quantiles. Under an exact Gaussian model, zero-threshold one-bit MSE-optimal levels would instead be $\pm\sigma\sqrt{2/\pi} \simeq \pm 0.79788\sigma$. For fixed signs and positive scale, however,

$$D_{\mathrm{P}} = \|q'\|^2 + da^2 - 2a\langle q', s\rangle, \qquad s \in \{-1, +1\}^d. \tag{2.6}$$

Thus scale cancels from primary ordering but still affects residual reconstruction. Our design uses separability and measured quality, without claiming optimal quantisation.

D_{raw} uses original vectors; $D_{\mathrm{P}} = \|q' - \widehat{p}\|^2$ uses primary reconstructions. The refined score is

$$D_{\mathrm{H}} = \|q' - \widehat{p} - \widehat{r}\|^2 + \|q'\|^2. \tag{2.7}$$

Its query-only term changes distance values, not ranking. Routing and reconstruction errors remain after exact computed-score selection.

2.4 GPU Execution Concepts

CUDA uses 32-thread warps and blocks with limited registers and shared memory [25]. JHQ spreads scanning across candidates and refinement across coordinates of one survivor. More blocks can hide latency, but also duplicate tables and buffers.

Faiss uses warp selection [18]; bitonic, radix and merge primitives trade work for state. AIR top-k and GridSelect study global selection [38]. JHQ adapts these existing tools.

CUDA Graph replay saves launch setup. Capture, workspace changes and shared-key access still follow the lifecycle and correctness contracts in Chapter 5.

2.5 Graph Search on GPUs

SONG exposes distance parallelism in GPU graph search [39]. CAGRA designs GPU construction and navigation, tuned by connectivity and search width [28]. We test both fp32 and quantised-vector CAGRA because their speed and storage differ.

BANG pairs GPU compressed vectors with a host-resident graph [37]. This trades transfer costs for capacity. JHQ's index remains device-resident.

E1 compares graph navigation with regular scans at batch 1,000; T5 tests their small-batch behaviour (section 7.2.5).

2.6 Quantised Search and Fast Tables

2.6.1 Learning a representation versus exploiting its structure

OPQ learns rotation and subspace codebooks [13]; this can remove the Cartesian structure needed for JHQ's exact split. ScaNN weights error for inner-product ranking [15]. Both show why reconstruction error alone is insufficient; Chapter 8 tests this within JHQ.

2.6.2 Small lookup tables have substantial prior art

PQ Fast Scan and Quicker ADC already shape tables for CPU register lookup [1, 2]. JHQ exploits Cartesian structure on a GPU, with separate checks for FP32 rounding and cutoff ties.

RaBitQ analyses a randomised compact-code estimator [11]; multi-bit variants trade storage for accuracy [12]. Its guarantees do not transfer to JHQ's primary/residual hierarchy. GPU IVF-RaBitQ supplies fused GPU search [33] and is a direct baseline.

2.6.3 Refinement effort and adaptive stopping

Faiss and cuVS expose refinement controls [7, 10]. Alternatives include GT-based autotuning [9], learned termination [19] and recall-targeted DARTH [6]. Chapter 6 instead selects one reusable budget from unlabelled reference agreement.

2.7 Research Gap and Design Choices

Table 2.1 compares the alternatives by the constraint they address. The literature already provides GPU graph search, compressed distance computation, register-oriented lookup, fast selection and adaptive effort. A useful contribution must therefore specify the representation contract, the integration decision and the evidence for it. "A GPU version" alone does not identify that contribution.

Table 2.1. Design alternatives and the thesis's narrower research gap. This is a mechanism comparison; measured rankings follow in Chapter 7.

Alternative	Useful structure	Principal trade-off	JHQ-GPU question
IVF-PQ / OPQ	Learned subspace codes; ADC	Distortion and training; arbitrary subcodebooks couple coordinates	What does an internally Cartesian primary code permit in GPU scoring?
Fast Scan / Quicker ADC	Small tables and SIMD lookup	Table precision, shape and ISA constraints	Does exact splitting repay extra GPU lookups?
SONG / CAGRA	Selective graph navigation	Dependent access, graph storage and construction	Where does a resident hierarchical scan compete?
BANG	Compressed GPU scoring with host graph	Capacity versus communication	What can complete compressed residency achieve?
IVF-RaBitQ	GPU code construction and fused search	Bit budget, estimator and probe effort	Which recall regions justify the residual hierarchy?
Adaptive effort	Refine ratios, autotuning, learned or declarative stopping	Labels, reference fidelity, prediction or calibration cost	Can unlabelled workload agreement select useful effort at an acceptable risk and total cost?

The open questions are practical: can this hierarchy support fast, correct GPU execution; when does extra scan parallelism pay; and does unlabelled budget calibration preserve enough quality to repay its cost? We keep the inherited representation fixed to study those choices.

3 Research Design and Measurement

We use paired execution tests, held-out calibration and tuned system frontiers. Each needs different controls. This chapter defines them; Chapters 4–6 describe the methods and Chapter 7 reports results.

3.1 Research Questions and Comparisons

Table 3.1 defines the comparison unit and what would challenge each research claim. A *cell* fixes a dataset, index configuration and workload setting; arms are alternative implementations or budgets within that cell. For paired execution measurements, the index state, queries, probe count, refinement budget and scoring representation are held fixed unless the named treatment changes them. The recorded paired ratio is $Q_{\text{treatment}}/Q_{\text{control}}$. Geometric means summarise multiplicative ratios over the stated cells, giving each cell equal weight.

Table 3.1. Research questions as testable comparisons. A capacity observation has a different control from a paired execution intervention.

Question	Intervention / comparison	Held fixed or accounted for	Measure and counter-evidence
RQ1: build and capacity	GPU construction; cached versus cold lifecycle; compressed versus raw storage	N, d, representation, routing and timed construction boundary	Train/add time and resident bytes; failed builds or model–allocation gaps delimit feasibility
RQ2: execution	LUT grouping, word access, refinement lanes, grouped scan, shared bound and probe selection	Paired state, workload and score; reference output semantics	Wall QPS, stage time and output checks; slowdowns or non-tie mismatches challenge the mechanism
RQ3: budget	Sampled / full-pool rule versus fixed 64, 100, 200, enumeration and GT oracle	One index and route; disjoint calibration and held-out query positions	Held-out recall loss, selected-arm QPS and calibration cost; threshold crossings challenge reliability
RQ4: operating region	Independently tuned JHQ, CAGRA, IVF-PQ and IVF-RaBitQ	Exported vectors, GT, k, device and host-to-host boundary; memory reported separately	Matched-recall QPS and coverage; stronger competitors at other recall targets challenge broad superiority

Paired ablations measure an implementation change. Explaining it through cache misses or occupancy would need hardware counters. Cross-system frontiers compare whole systems; the integrated scheduling test in section 7.6 avoids multiplying gains from different baselines.

The E1 grid was extended where recall coverage was incomplete. Low-recall losses, high-recall reversals and unsuccessful alternatives remain part of the evaluation. The rule uses reference agreement; held-out evaluation measures raw-vector GT recall.

3.2 Datasets, Queries, Ground Truth and Splits

Table 3.2 records the six exported datasets used by the newer budget, LUT and scale campaigns. The poster's E1 comparison uses the first four. Vogue contains fashion-image embeddings; arXiv contains abstract embeddings; the two OpenAI datasets use Qdrant's DBpedia exports [5, 31, 32, 34]. BGE uses the Italian Wikipedia configuration, and Stella uses the TREC24 BioGen corpus export [35, 36]. Sizes describe the measured exports after preparation, rather than the current upstream cards.

Table 3.2. Data and JHQ training configuration. n_q is the actual query count; $n_t = 39n_{\text{list}}$ is the requested coarse-training sample size. E1 uses the first four rows. All rows use squared L2 and $k = 10$.

Dataset	N	d	n_q	n_{list}	n_t
Vogue	932,328	768	1,000	4,096	159,744
arXiv	2,253,000	768	1,000	8,192	319,488
OpenAI-1536	999,000	1,536	1,000	8,192	319,488
OpenAI-3072	999,000	3,072	1,000	8,192	319,488
BGE-M3	10,091,524	1,024	1,000	32,768	1,277,952
Stella	17,776,615	1,024	999	32,768	1,277,952

OpenAI/BGE/Stella preparation removes non-finite rows, normalises norms above 10^{-12} and samples queries without replacement using seed 42. Stella supplies 999 corpus-sampled queries, not its separate 65-query benchmark. These exports test vector neighbours, not relevance labels. The OpenAI exports share the sampling rule; checks support row alignment on all queries and 10,299 base rows, without original-ID joins (sections 7.2 and 7.2.3).

All methods in a cell read the same query and GT files. Ground truth is computed by exhaustive squared-L2 search over exported FP32 vectors; the streamed converter maintains top-k across database tiles. Evaluation uses stored GT order, including its floating-point tie choices. With $\mathcal{T}_k(q)$ the first k GT IDs and $\mathcal{R}_k(q)$ the returned ID set,

$$\text{Recall}\,@k = \frac{1}{|\mathcal{Q}|k} \sum_{q \in \mathcal{Q}} |\mathcal{R}_k(q) \cap \mathcal{T}_k(q)|. \tag{3.1}$$

GT rows are 100 wide for Vogue/arXiv and 20 wide for the other four; only ten entries are scored. Duplicate returned IDs do not earn repeated credit. Range, duplicate and repeat-call checks are recorded separately.

JHQ uses $M = d/8$, $B = B_r = 8$, eight coarse iterations and 100,000 residual-training rows. Sweeps reuse trained states but re-encode the database; cold builds start afresh. Training uses database rows. Even/odd query positions give calibration/held-out pools of 500/500, or 500/499 for Stella. Seeded subsamples of sizes 32, 64 and 128 are drawn 64 times per cell; FULL uses all 500 calibration queries. GT labels enter the offline oracle and held-out assessment, while the deployable agreement rule uses search outputs (sections A.3 and 6.2).

Preparation code and shape logs establish this protocol; complete historical vector-file and trained-state hashes are unavailable.

3.3 Timing, Variance and Memory Accounting

The principal comparison starts with FP32 queries in host memory and ends when top-10 IDs and scores are available in host memory after stream completion (figure 3.1). It includes pinned staging, H2D, query preprocessing, search and D2H. CAGRA int8 query quantisation is inside this boundary; quantiser training and index construction are outside it. Resource handles and output buffers are reused. JHQ returns to pageable host arrays and the baselines to pinned arrays; CAGRA/PQ use Python APIs, whereas JHQ/RaBitQ use C++. These interface differences remain visible when interpreting small margins.

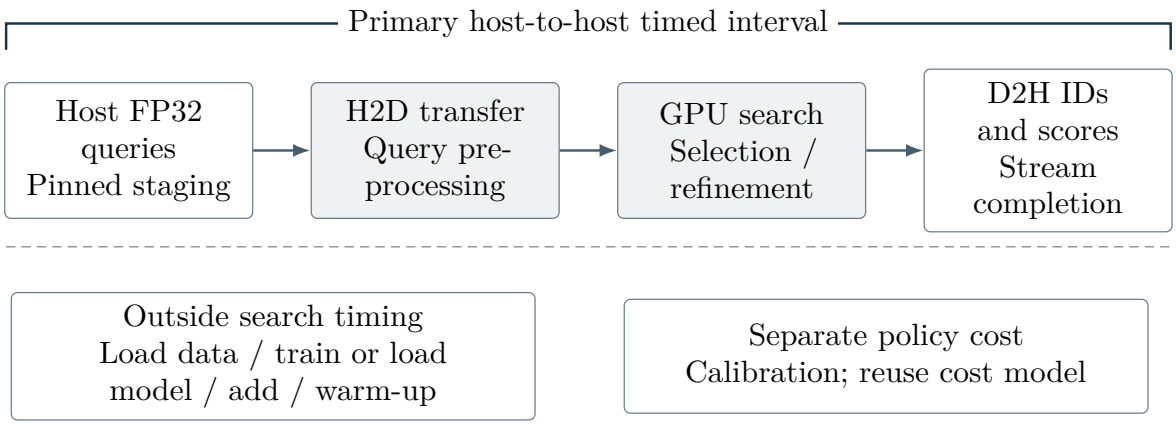


Figure 3.1. Timing boundaries. The host interval includes transfers and query-specific computation. Device-stage profiles explain parts of that interval; separate profiling and synchronisation can change execution, so their sum is not an interchangeable end-to-end latency.

With t in seconds, round throughput and its summary are

$$Q_r = \frac{m_r b}{\sum_{i=1}^{m_r} t_{ri}}, \qquad Q = \operatorname{median}_r Q_r, \tag{3.2}$$

E1 uses batch 1,000 and five rounds. JHQ warms up once per variant/round; baselines warm up three times initially. Normally each round has 24 timed calls, fewer for slow baseline points. Warm-up excludes JIT and capture. Latency percentiles describe repeated batch calls, not individual or concurrent requests.

We retain slow valid calls and both round-QPS and call-latency summaries. Their medians need not be reciprocal. Shared indexes and query pools do not provide independent training replications, so sweep cells receive no independence-based significance claim.

Runs use an RTX 5090 (32,607 MiB), driver 595.71.05, CUDA 13.0 and cuVS 26.08.01. OpenMP uses 16 threads within a 25-CPU quota; E1 records 575 W. A cooperative lock and snapshots do not fix clocks or host load. JHQ JIT-compiles Release compute_80 PTX [27]; baselines use installed builds. Device profiles are reported separately.

Resident memory includes encoded arrays, models, workspace and context. E1's context-subtracted JHQ totals and CAGRA's index deltas therefore differ in scope (sections 4.4 and 7.7). Later-created search pools and build temporaries prevent an equal-peak-memory comparison. Separate 20 ms samples estimate build peaks. Cold train/add excludes input loading and preceding host setup.

Supplementary evidence boundary. Supplementary checks (section A.4) use separate sessions. Precision changes and cached-model reuse follow sections 4.2 and 7.2.3; historical baseline ratios remain cross-session.

3.4 Baselines and Frontier Construction

The competing families cover graph traversal (CAGRA fp32/int8), product-quantised IVF and GPU IVF-RaBitQ [17, 28, 33]. FP32 GPU brute force provides an exhaustive-search reference. Each family sweeps its own implementation parameters (table 3.3); the recorded index configurations can differ in storage and reconstruction fidelity. Consequently the main frontier measures achievable speed at a quality target within these grids. A separate memory account accompanies it.

Table 3.3. Principal E1 tuning dimensions. Exact per-point parameters and coverage extensions remain in the raw records. The grid is finite and differs between methods.

Family	Recorded search dimensions
JHQ	Probes and α; initial probes 64/128/256 and 0.5/1/2 times the workload budget; coverage extensions reach probe 1,024 and larger/smaller budgets
CAGRA fp32/int8	Graph degree 32/64; search width 1/2/4/8; intermediate top-k capacity 32–512
IVF-PQ	Eight-bit subcodes; 192/384/768-byte codes at $d = 768$, 384/768/1,536 at higher dimensions; fp16/fp32 lookup settings; probes 16–512
IVF-RaBitQ	4/8/9 bits per dimension, supported search modes, probes 16–1,024; additional 4,096-list OpenAI indexes
GPU brute force	Exhaustive FP32 squared-L2 search, common host boundary

A point dominates another if recall and QPS are both no worse and one is better. The frontier can span index configurations. At target r, the measured estimate is

$$Q_{\text{meas}}(r) = \max\{Q_i : R_i \geq r\}. \tag{3.3}$$

It is undefined without a qualifying point. For adjacent frontier points, interpolation uses

$$\log Q_{\text{int}}(r) = (1-\lambda)\log Q_a + \lambda \log Q_b, \qquad \lambda = \frac{r - R_a}{R_b - R_a}. \tag{3.4}$$

The parser first accepts a point within 5×10^{-4} of the target; otherwise it interpolates inside the measured range only. T5 also keeps qualifying measured endpoints below a method's minimum frontier recall. We report both estimates near ties; neither matches storage budgets.

Failures and invalid outputs stay outside the valid frontier. Repeat-call set and order changes are logged separately. Isolated reruns replace contaminated builds; skipped Stella batch-1,000 runs supply no points (section A.2). Current frontiers include int8 CAGRA (figure 7.1); historical ADC/poster plots retain their original scope.

3.5 Experimental Records and Reproducibility

Each family records version, index/query settings, timing and outputs. Paired tests share trained states; state or precision changes define a new family. Missing historical executable/model fingerprints limit exact reproduction (appendix A).

3.6 Evidence Strength and Threat Controls

Correctness refers to stated computed scores and tie rules. Raw recall also depends on routing and reconstruction. Sampling/order conditions are in sections 6.2 and 6.3; section 8.3 covers finite pools, shared indexes, tuning grids, interfaces and hardware limits.

4 GPU Index Design and Construction

GPU construction trains and encodes JHQ's representation into list-ordered arrays with bounded temporary storage [16]. Query kernels and measured construction costs are in Chapter 5 and section 7.7.

4.1 End-to-End Build and Search Boundaries

Training creates reusable models; add encodes rows and builds lists; search uses the completed index. A cache hit skips fitting, not database encoding. Cold train-plus-add and cached-state search therefore measure different work.

The host prepares samples, the square transform and primary codebook. GPU products apply rotation; device training fits IVF centroids and residual codebooks. Add then transfers bounded batches, encodes both levels and writes list-ordered arrays.

Model state holds the transform, codebooks and centroids. Database state adds offsets, IDs, codes and corrections. Queries overwrite workspace but reuse its allocations; a captured graph references those existing arrays.

One successful `add` creates a fixed snapshot; another add is rejected. Updates require a replacement snapshot. Physical positions stay fixed during search; mutable insert/delete and transactional publication are unimplemented.

Search maps host queries to host IDs and computed scores. Equation (5.20) explains the query-only score term; section 3.3 defines timing.

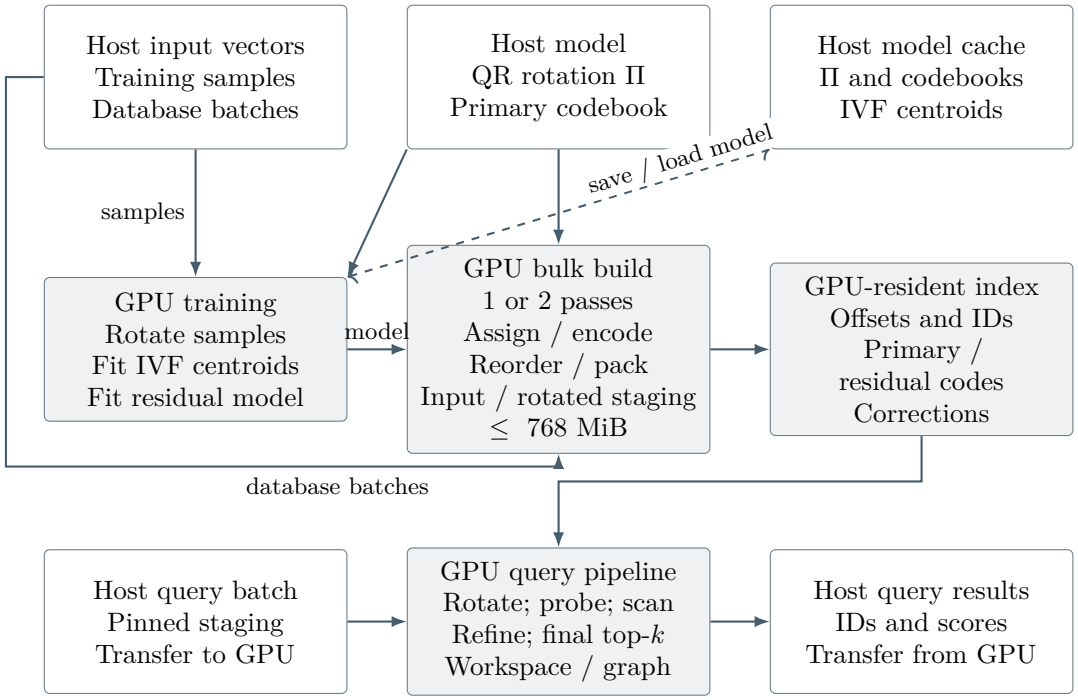


Figure 4.1. Implemented build and query boundaries. Host work includes sampling, orthogonal-matrix construction and analytical primary-codebook generation; the evaluated configuration fits IVF and residual models on the GPU. The cache stores trained state, whereas the completed database representation resides on the device. Dashed arrows denote trained-state reuse. The staging label bounds only raw/rotated float buffers at source defaults; other pipeline scratch is additional.

4.2 Training, Rotation and Codebook Generation

4.2.1 Transform and sampling

The implementation draws a seeded Gaussian matrix on the host and uses LAPACK QR routines to construct a square orthogonal matrix Π. For row-major training data X_t, device matrix multiplication computes

$$Y_t = X_t\Pi^{\mathsf{T}}, \qquad y = \Pi x, \qquad \|\Pi(q-x)\|_2^2 = \|q-x\|_2^2 \quad \text{when } \Pi^{\mathsf{T}}\Pi = I. \tag{4.1}$$

The output retains dimension d. This is an orthogonal change of coordinates; QR construction and device products have finite-precision error, whose measurement is part of C0 in section 4.6.

The harness selects database rows at a fixed stride. The primary/IVF sample has 159,744 rows for Vogue, 319,488 for arXiv and both OpenAI datasets, and 1,277,952 for BGE and Stella: exactly $39n_{\text{list}}$ in each case. The $39n_{\text{list}}$ rule matches FAISS's warning threshold [8], without guaranteeing fit quality. A separate 100,000-row strided sample bounds residual-training cost.

An earlier sweep fixed primary/IVF models and varied residual samples on Vogue/OpenAI-3072 ($B_r \in \{4, 8\}$, 32/128/256 probes). Across twelve configurations, 100K rather than all rows lowered recall in eleven cases and tied once; the largest gap was 0.0018. Intermediate sizes were non-monotone. This 25-Lloyd-iteration study motivates a bounded sample, not the later six-dataset result.

4.2.2 Why retain the analytical primary model?

The constructor estimates scale as

$$\sigma^2 = \frac{1}{n_t d}\sum_{i=0}^{n_t-1}\|x_i\|_2^2, \qquad D_s = d/M, \qquad K = 2^B. \tag{4.2}$$

For $B = D_s = 8$, levels are $\{-a, +a\}$ with $a = 0.674\sigma$: one byte stores eight signs. JHQ's analytical structure [16] allows $O(d)$ encoding and exact small-LUT factorisation (section 5.2). Arbitrary Lloyd-refined codewords need not remain separable.

For fixed signs $\widehat{p} = as$, $s_j \in \{-1, +1\}$,

$$\|q' - as\|^2 = \|q'\|^2 + da^2 - 2a\langle q', s\rangle. \tag{4.3}$$

Positive a does not change real-arithmetic primary order, without a Gaussian assumption. It does change residuals and refined ranking. Zero scale is degenerate; FP32 ties still matter.

On historical Vogue ($M = 96$, $B_r = 4$, 128 probes), earlier/analytical-primary builds gave primary recall 0.6618/0.6342 and full recall 0.9452/0.9409. Refinement narrowed the gap. Residual training also changed, so this is not a primary-codebook-only experiment. Chapter 7 tests the complete frontier.

4.2.3 Coarse assignment as filter and refine

Eight IVF iterations update centroids seeded from spaced rotated rows. Assignment ranks $s_\ell = \|\mu_\ell\|^2 - 2\langle y, \mu_\ell\rangle$. Under supported dimensions, the default filters with int8 products and resolves ambiguity in FP32; other precision paths also exist.

Lemma 4.1 (Assignment with certified score intervals)**.**
Let $\widetilde{y}, \widetilde{\mu}_\ell$ be dequantised approximations, $e = y - \widetilde{y}$ and $f_\ell = \mu_\ell - \widetilde{\mu}_\ell$. Keeping centroid norms exact, the approximate score $\widetilde{s}_\ell = \|\mu_\ell\|^2 - 2\langle \widetilde{y}, \widetilde{\mu}_\ell \rangle$ has real-arithmetic error bounded by

$$E_\ell = 2(\|y\|\|f_\ell\| + \|e\|\|\widetilde{\mu}_\ell\|). \tag{4.4}$$

Given valid intervals $[L_\ell, U_\ell]$ containing every target score, discarding only $L_\ell > \min_j U_j$ and exactly resolving the remaining scores preserves the minimum and its specified tie rule.

Proof.
Expand the dot-product error as $\langle y, f_\ell \rangle + \langle e, \widetilde{\mu}_\ell \rangle$ and apply Cauchy–Schwarz. A discarded score exceeds an upper bound on some retained score, so it cannot attain the minimum. Keeping interval equality preserves possible ties. □

The filter uses conservative norm bounds and FP32 headroom. A bitwise FP32 guarantee would also need accumulation-error bounds. The archived FP16 comparison differs on 0/14,838 Vogue, 6/31,748 BGE and 3/16,359 Stella tail assignments. This is finite evidence; cold logs do not identify the executed assignment precision.

4.2.4 Residual fitting and the iteration budget

Residual fitting follows IVF but uses $r = \Pi x - \widehat{p}(x)$, not an IVF-centroid residual. Each subspace shares a sorted $K_r = 2^{B_r}$ scalar codebook across its coordinates. GPU sorting and one-dimensional Lloyd updates fit it, capped at 2,000 iterations.

At the historical 100K sample, 2,000 rather than 25 updates gained 0.0063–0.0076 recall. Later changes exceeded reported two-run variation in BGE/Stella, and sparse tail bins kept changing. Cold runs hit the cap. This is budgeted FP32 fitting, not a claimed convergence result or a counterexample to exact Lloyd descent. At 20,000 updates, historical Stella all-row/100K recall was 0.9921/0.9920.

4.2.5 Construction work and observed imbalance

Dense QR costs $O(d^3)$ once; dense rotation costs $O(n_t d^2)$. Direct coarse assignment costs $O(n_t n_{\text{list}} d)$ per iteration. Analytical primary construction writes $O(MKD_s)$ values; residual extraction touches $O(n_r d)$ coordinates, followed by sorting and scalar updates. These describe work, rather than predict latency. Synchronised phase diagnostics place about 83% of BGE/Stella training in IVF k-means (about 2.97 s). They use one add buffer and forced synchronisation, separately from the normal overlapped timings in section 7.7.

List-size imbalance enters the scan rule in section 5.5. The individual contributions of eight IVF iterations, spaced seeds and finite sampling to that imbalance were not isolated.

Arithmetic used in the measured implementation. The evaluated implementation (v69) stores FP32 matrices but enables cuBLAS TF32 tensor-core math for $d \geq 1{,}536$. In its compiled faithful branch, a null override falls through to the dimension-based default. This affects training/add/query rotation and centroid GEMMs on both OpenAI datasets; an environment override cannot disable that branch. Thus FP32 storage does not imply full FP32 multiplication. The supplementary FP32 variant (v72) corrects this switch. Its sensitivity experiment reuses the same TF32-trained cache and changes add/query arithmetic, rather than retraining the model in FP32 (section 7.2.3).

4.3 IVF Assignment, IDs and Physical Memory Layout

4.3.1 From database IDs to list-ordered positions

An input row has identifier $i \in \{0, \ldots, N-1\}$. Coarse assignment places its transformed vector in a list according to the computed centroid scores. The mathematical assignment

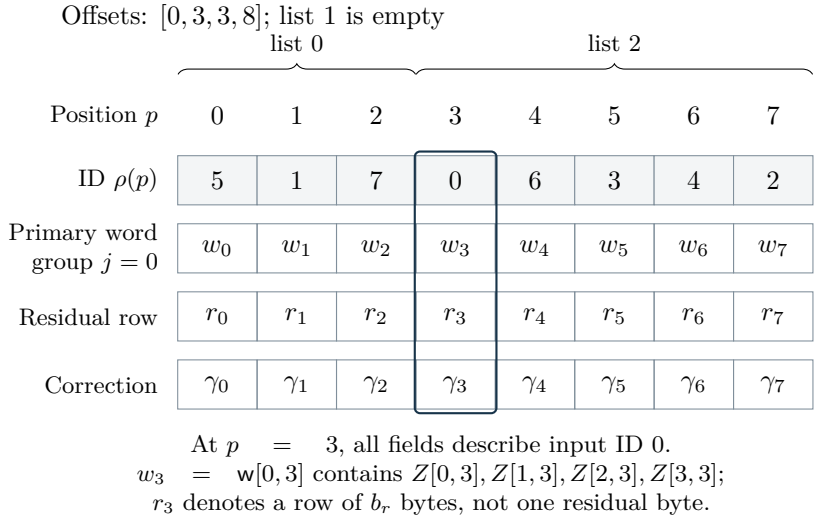


Figure 4.2. One common physical position joins IDs, packed primary words, residual rows and correction factors. This illustrative eight-row index has offsets $[0, 3, 3, 8]$, including an empty list. A primary word packs four subspaces of one vector. Values and list sizes are schematic.

target is

$$a(i) = \operatorname*{arg\,min}_{0 \le \ell < n_{\text{list}}} \|\Pi x_i - \mu_\ell\|_2^2. \tag{4.5}$$

Its implementation uses the selected numerical assignment path. Encoding the primary and residual representations does not subtract $\mu_{a(i)}$; list membership restricts the future search universe independently of those reconstructions.

The builder sorts assignments together with a sequence of input IDs. Let $\rho(p)$ be the input ID at sorted physical position p and let o_ℓ be the start of list ℓ. The completed arrays satisfy

$$0 = o_0 \le o_1 \le \cdots \le o_{n_{\text{list}}} = N, \qquad p \in [o_\ell, o_{\ell+1}) \Longrightarrow a(\rho(p)) = \ell. \tag{4.6}$$

Offsets are obtained by lower-bound searches over the sorted assignments. Equal consecutive offsets represent an empty list. Sorting by assignment does not promise stable ordering of equal-list rows across independent builds. A position is stable within the completed snapshot; its meaning is the row $\rho(p)$ shared by every associated array.

Resident IDs map positions p back to input rows $\rho(p)$. Two-pass add builds the inverse temporarily for scatter; search needs no permanent inverse. Primary ties use positions, final ties use database IDs.

4.3.2 Exact addresses and representation widths

Let $Z[m, p]$ denote the primary byte for subspace m at physical position p. The intermediate subspace-major copy has byte offset $mN + p$. Final packing produces

$$\mathsf{w}[j, p] = \sum_{t=0}^{3} 2^{8t} Z[4j + t, p], \qquad \text{addr}_{\text{byte}}(\mathsf{w}[j, p]) = 4(jN + p). \tag{4.7}$$

Here $j = 0, \ldots, M/4 - 1$. Word layout requires $M \bmod 4 = 0$; the active word-layout build rejects other values. A separate byte-layout compilation retains the byte representation. Each packed array occupies NM bytes, but repacking temporarily holds both old and new copies, costing $2NM$. The one-pass builder temporarily gathers a vector-major $[N, M]$ primary array, transposes it, and then repacks; each obsolete copy is freed. The two-pass builder scatters

Table 4.1. Per-vector bytes. JHQ includes codes, ID and correction; SQ8 is vector codes alone; CAGRA int8 models vector codes plus 32 four-byte neighbours. Offsets, model parameters, workspace and other runtime allocations are excluded. These components are not complete process totals.

d	Primary	Residual	ID + corr.	JHQ	SQ8	CAGRA int8	FP32
768	96	768	8	872	768	896	3,072
1,024	128	1,024	8	1,160	1,024	1,152	4,096
1,536	192	1,536	8	1,736	1,536	1,664	6,144
3,072	384	3,072	8	3,464	3,072	3,200	12,288

directly into the subspace-major array before the same final repack. The search-time lane mapping is discussed in Section 5.3.

Residual codes remain row-major, with $b_r = \lceil dB_r/8 \rceil$ bytes per vector. In the eight-bit configuration, coordinate t of row p is at byte offset $pb_r + t$. In the four-bit path it is in byte $pb_r + \lfloor t/2 \rfloor$, with even coordinates in the low nibble and odd coordinates in the high nibble. A four-byte correction per physical row stores

$$\gamma[p] = 2\langle \widehat{p}(x_{\rho(p)}), \widehat{r}(x_{\rho(p)}) \rangle. \tag{4.8}$$

Its value must move with that row's codes during gathering or scattering. Residual access then reconstructs the same object that primary screening identified, even though the two code arrays use different layouts.

This separation favours the workload's two access patterns: primary codes are scanned broadly in list order, while complete residual rows are read for selected positions. Retaining raw FP32 vectors on the device would provide exact reranking but consume $4Nd$ additional bytes and change the reported search design. This index instead refines compressed reconstructions, with the quality consequences evaluated in Chapter 7.

4.4 Resident Storage, Workspace and Index Lifecycle

4.4.1 Stored bytes and allocated bytes

With one-byte primary subspace codes, resident row and offset bytes are

$$S_{\text{rows}} = N\left(M + \left\lceil \frac{dB_r}{8} \right\rceil + 8\right), \qquad S_{\text{offsets}} = 4(n_{\text{list}} + 1). \tag{4.9}$$

IDs and FP32 corrections add eight bytes per row. At $M = d/8$, $B_r = 8$, codes take $9d/8$ bytes, 12.5% above SQ8. Graph-free storage is not a unique compression advantage over int8.

Degree-d_g CAGRA uses $d + 4d_g$ bytes per row. For Stella at $d_g = 32$, JHQ's 19.20 GiB payload nearly equals CAGRA int8's recorded 19,656 MiB, despite different accounting scopes. JHQ instead offers graph-free construction and adjustable residual access; α does not shrink the stored array.

Basic FP32 arrays take $4d^2$ transform bytes, $4MKD_s$ primary-codebook bytes, $4MK_r$ residual-codebook bytes and $4n_{\text{list}}(d+1)$ centroid/norm bytes. The selected path adds derived arrays. Workspace holds two query matrices $[b_{\text{cap}}, d]$, centroid scores, LUTs, probes and survivors. Optional residual LUTs add $4b_{\text{cap}}dK_r$ bytes; defaults score directly. Group scratch (equation (5.13)) reserves about 93.8 MiB at 128 probes, even on the query-centric path.

Input remains caller-owned. Build staging uses $4n_b a_{\text{in}} d$ pinned-host bytes. Order/inverse arrays each take $4N$ device bytes. Assignment scratch is freed before gather (equations (4.10) and (4.13)).

Table 4.2. Analytical row-array occupancy (GiB), $d = 1{,}024$, $M = 128$, $B_r = 8$. BGE uses one pass; Stella uses two. The one-pass Stella column is counterfactual capacity analysis. Pipeline, model, offsets and runtime costs are excluded from these phase values.

Dataset	Final row payload	One-pass gather	Two-pass repack
BGE	10.90	23.05	12.11
Stella	19.20	40.59	21.32

4.4.2 Reuse, recovery and invalidation

The cache stores rotation, codebooks and centroids. Its key omits code version and full database hashing, so changed code/data require explicit invalidation and model provenance. Sweeps reuse states; cold builds create new ones. Similar recall does not imply identical models.

A cache hit still performs constructor QR, warm-up, model upload and add. Stella's 6.67 s add record includes initial workspace allocation, but excludes schedule calibration and search warm-up. It is not a restart-time measurement.

Changing α needs no retraining. Before capture, search grows workspace as needed; changes to batch, survivors, probes, lanes, groups or bounds trigger recapture. Calibration can bypass capture. The index owns and reuses buffers across sequential calls, then releases them with its graph.

4.5 Construction Cost and Tested Scale

4.5.1 Bounded input batches and two-pass construction

Rotating all N raw vectors at once requires two FP32 device arrays, together $8Nd$ bytes. The builder instead stages bounded batches through pinned host memory and device raw/rotated buffers. For n_b pipeline buffers, configured add rows a_{add} and per-array byte budget S_{stage}, the source stages

$$a_{\text{in}} = \max\left\{1, \min\left(a_{\text{add}}, \left\lfloor \frac{S_{\text{stage}}}{4d} \right\rfloor\right)\right\}, \qquad S_{\text{float}} = 8n_b a_{\text{in}} d. \tag{4.10}$$

Three buffers and the default 128 MiB budget give at most 768 MiB of input/rotated device floats. Assignment products and encoder scratch are separate allocations.

One-pass add encodes input-order arrays, then gathers them into list order, temporarily keeping both copies. Two-pass add assigns/sorts IDs first, then revisits input to encode and scatter directly. It trades another read/rotation for lower simultaneous storage. Raw-input assignment against back-transformed centroids uses the real-arithmetic orthogonality identity.

4.5.2 Phase occupancy and the process peak

The one-pass capacity estimate is

$$H_{\text{single}} = N(2b_r + 3M + 20) + 2^{30} \quad \text{bytes.} \tag{4.11}$$

Ten live gather/transpose arrays contribute $3NM + 2Nb_r + 20N$ bytes: three primary copies, two residual/correction copies, assignments, sort order and IDs. The extra 1 GiB is heuristic headroom, not a full peak model.

At final two-pass repack, IDs, residuals, corrections and both primary layouts occupy

$$S_{\text{repack},2} = N(b_r + 2M + 8). \tag{4.12}$$

This counts that phase only. Whole-build peak also considers earlier pipeline/inverse scratch, model, sort, context and later workspace allocations.

Table 4.2 quantifies the compressed-copy saving. Stella's cold log selects two passes and prints 41.6 GiB for equation (4.11), against 30.6 GiB available at the tail. The separately

sampled resident process totals, including context and query workspace, are 11.93 GiB for BGE and 20.44 GiB for Stella (section 4.4). The supplementary 20 ms device samples observe a 26.08 GiB BGE peak near the start of add, and a 22.58 GiB Stella peak during repack. These are sampled device totals including context; shorter-lived peaks can be missed.

Assignment-product capacity uses an independent configured row count a_{assign}, selected by `JHQ_ASSIGN_BATCH` or the source default: 32,768 for at least 8,192 lists, otherwise 8,192.

$$S_{\text{dots}} = 4 n_b n_{\text{list}} a_{\text{assign}} \quad \text{bytes.} \tag{4.13}$$

Allocation capacity need not shrink with a_{in} or the rows in a particular call. Table B.3 checks four defaults: the 1,536/3,072-dimensional cases allocate 3 GiB, despite different staging rows. BGE/Stella still allocate three 4 GiB arrays, or 12 GiB, near add entry. The 23.05 GiB gather payload describes a later lifetime: adding 12 GiB to it would not give the actual peak. Peak modelling must take the maximum over stage-specific live arrays, including pipeline scratch. Stella's smaller two-pass repack payload, plus model/context, explains its lower observed peak despite its larger database.

4.5.3 Construction measurements

4.6 Index Representation and Safety Contracts

A historical 32-bit $n_t d$ overflow at 2^{31} zeroed the primary scale; a later process reloaded it from a version-blind cache. Chunked norms with double accumulation fixed it. Routing alone therefore cannot validate representation.

4.6.1 Completed representation evidence

The 13 October pilot exported cached models and decoded layouts for full Vogue (932,328), arXiv (2,253,000) and OpenAI-3072 (999,000) databases.

Codebooks passed separability, scalar-level and sorting checks. An FP64 loop checked $\gamma = 2\langle \widehat{p}, \widehat{r} \rangle$ on every exported row; maximum absolute error was 1.28×10^{-7}–2.77×10^{-7}. Sampled GPU-returned top-10 scores have maximum relative errors from 2.38×10^{-7} to 4.85×10^{-5}, using $|s_{\text{GPU}} - s_{64}|/s_{64}$ with the query-norm constant included. These are checks of the exported representation, not re-encoding from the original input vectors.

Across seven workloads and 896 query instances, the pilot additionally checked the proposed residual bounds against FP64 reconstructed distances for 72,800,590 routed candidate instances, with zero reported violations. The same population supports the FP64 composite-score identity check. GPU-returned-score checks cover only top-10 slots and skip IDs absent from the routed map, leaving output membership incompletely checked.

Encoder, primary/residual training and construction sources match the evaluated implementation after namespace normalisation and removal of the export method. This source correspondence covers the shared path, leaving later model identity and large-dataset two-pass construction to separate checks.

4.6.2 Independent input and placement audit

The supplementary exporter audit uses the shared construction path on Vogue, arXiv, OpenAI-3072 and Stella. Offsets are monotone with endpoints 0 and N; IDs form a permutation of every database row in all four. FP64 products of the stored rotation give maximum entrywise $\|\Pi\Pi^{\mathsf{T}} - I\|$ errors of 3.96–4.34 $\times 10^{-7}$. Sampled norm checks use FP64 rotation, rather than the GPU GEMM, and therefore measure the stored matrix's geometry. Table 4.3 summarises sampled recoding.

Table 4.3. Original-input FP64 recoding audit. Flips count coordinate decisions; the final column is the largest residual-boundary distance among mismatches. Structural checks cover all rows; recoding is sampled.

Dataset	Rows	Orth. error	Primary flips	Residual flips	Max margin
Vogue	200,000	4.11e-07	14	5,409	1.12e-07
arXiv	100,000	4.11e-07	1	2,515	7.31e-08
OpenAI-3072	20,000	3.96e-07	5,745	1,646,806	2.67e-05
Stella	100,000	4.34e-07	16	3,064	1.03e-07

At 768/1,024 dimensions, recoding mismatches fall within 1.12×10^{-7} of boundaries. OpenAI-3072 instead differs on 2.68% of residual coordinates, up to 2.67×10^{-5} from boundaries, consistent with TF32 rotation. Residual checks condition on stored primary digits to avoid double-counting flips.

ID-joined one-/two-pass builds have identical codes on the three smaller databases. Correction differences reach 8.94×10^{-8}; assignments differ on 1 Vogue, 1 arXiv and 40 OpenAI rows. Stella covers only its natural two-pass build. Exported centroids are absent, leaving membership decisions unchecked.

4.6.3 Contract coverage and remaining checks

Construction requires $M \mid d$, $D_s \mid B$ and $M \bmod 4 = 0$. It also needs finite inputs, representable counts and reserved sentinels. IDs/positions are signed 32-bit; address products use wider arithmetic.

T4 checks selection separately (section 7.4.1). Export audits do not certify centroid membership, empty lists or adversarial tails.

Table 4.4. Representation contracts after the pilot and supplementary input audit. Coverage separates stored geometry, sampled arithmetic and row placement.

Boundary	Required property	Evidence / remaining check
C0: geometry	Common transform with bounded floating-point isometry error	Stored-matrix orthogonality checked on four datasets; TF32 GPU arithmetic distinguished
C1: codebooks / correction	Cartesian levels, sorted residual codebooks, correction matches decoded codes	Pilot checks completed on three databases; all-row correction errors in section 4.6
C1: input encoding	Stored codes encode the intended original input row	FP64 original-input recoding on four samples; boundary and TF32 discrepancies quantified
C1: placement	Offsets cover rows; IDs form a permutation; all fields follow it	All-row offsets/ID checks; three ID-joined path comparisons; centroid membership unverified
Execution	Valid indexing, synchronisation and score-order contracts	Archived query and sanitizer checks; finite coverage, detailed in section 7.4

5 Representation-Aware GPU Search

The transform and primary/residual hierarchy follow JHQ [16]. This chapter develops their GPU scoring and computed-score selection; Chapters 6 and 7 cover measurement and budget choice.

5.1 Query Path and Four Distinct Ranking Objects

A query follows rotation $q' = \Pi q$, IVF routing, primary screening and residual refinement. Final selection returns k IDs and computed scores. Figure 5.1 distinguishes the three runtime selection decisions; the ground-truth ordering is an external evaluation object.

Four quantities must be kept separate:

$$\begin{aligned} s_i(q) &= \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, && \text{centroid routing score,} \\ D_{\mathrm{P}}(q,x) &= \|q' - \widehat{p}(x)\|_2^2, && \text{primary score,} \\ D_{\mathrm{H}}(q,x) &= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2, && \text{refined ranking score,} \\ D_{\mathrm{raw}}(q,x) &= \|q - x\|_2^2, && \text{raw-space distance.} \end{aligned} \tag{5.1}$$

Here μ_i is an IVF centroid in transformed coordinates, $\widehat{p}(x)$ is the primary reconstruction and $\widehat{r}(x)$ is the decoded residual. The runtime refined score includes a query-only constant, as derived from the implemented correction in Section 5.6. It preserves the ordering of reconstructed distances for a fixed query, but neither it nor the primary score is the original-vector distance.

Routing defines the visited universe U of n_c candidate occurrences; primary screening retains up to $c = \max\{k, \lceil \alpha k \rceil\}$ for residual scoring. Table 5.1 records each stage's inputs, reads and costs; sections 5.2 to 5.7 develop their implementations.

The ordering and exactness columns of table 5.1 attach each claim to its score and tie rule. Chapter 7 tests survivor sets, ordered outputs and raw-space recall separately.

Table 5.1 gives the stage contracts. The E1/budget harness explicitly selects 16 refinement lanes and scan policy -1 (scheduling among 0/2/4/8), versus library defaults of scalar refinement and policy 0. Shared bounds are enabled but used only at $g_{\mathrm{scan}} > 1$; the harness calibrates scheduling with 512 database pseudoqueries, a step the library does not perform automatically. Component ablations retain their stated overrides; table A.1 maps mechanisms to recorded implementations.

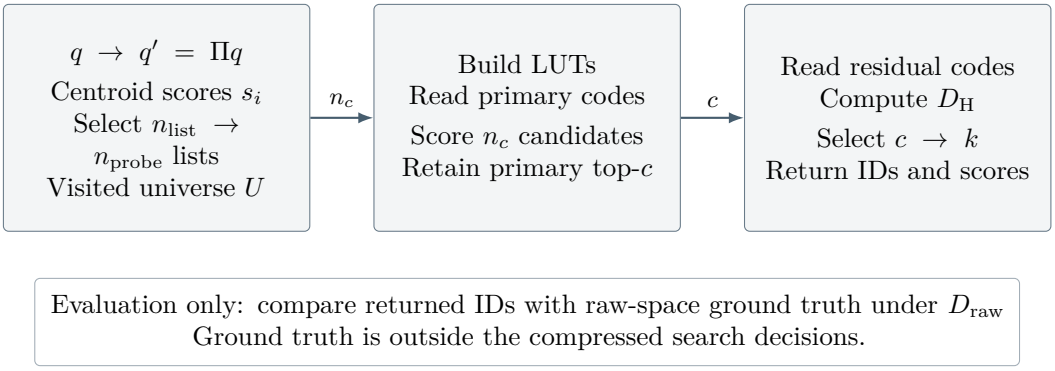


Figure 5.1. Query execution and its separate selection boundaries. Routing selects lists, primary screening selects vector candidates, and final selection ranks refined survivors. Raw-space ground truth is used for evaluation; it is not an input to these search decisions.

Table 5.1. Query-stage contracts. $c_v = \min(c, n_c)$ counts valid primary survivors. Exactness refers to the stated computed scores; routing and quantised reconstruction determine raw-space approximation.

Stage	Input → output	Data read	Main work	Order / exactness
Transform	$q \to q'$	Query; transform	Apply Π	Transform (precision in section 7.2.3)
Probe	$n_{\text{list}} \to n_{\text{probe}}$ lists	FP32 centroids; list lengths	Score; radix; rank count	(s_i, i); exact probe selection
LUT	$q' \to$ tables	Primary codebook	$MD_s 2^{8/g_{\text{LUT}}}$ terms	Real-arithmetic factorisation; FP32 rounding
Primary, $g_{\text{scan}} = 0$	$n_c \to c_v$	Packed primary codes; LUT	$O(n_c M)$ scoring; compaction	Exact distance-order selection; arbitrary cutoff ties
Primary, $g_{\text{scan}} \geq 1$	$n_c \to c_v$	Same primary representation	Score; compact; merge	(D_{P}, p); exact under stated premises
Refine	$c_v \to c_v$ scores	Residual codes; correction	$O(c_v d)$	Computed D_{H}; lane-dependent FP32 association
Final	$c_v \to k$	Refined scores; IDs	$O(kc)$ slot scans	$(D_{\text{H}}, \text{ID})$; exact with k finite valid entries

5.2 Exact Cartesian LUT Factorisation

Primary ADC keeps q' in FP32 and reconstructs stored codes [16, 17]:

$$T_m[z] = \|q'_m - \mathcal{C}_m[z]\|_2^2, \qquad D_{\text{P}}(q, x) = \sum_{m=0}^{M-1} T_m[z_m(x)]. \tag{5.2}$$

An eight-bit label normally needs 256 entries. The useful extra structure is inside each Cartesian subcodebook, beyond the PQ subspace partition.

At $K = 256$, $D_s = 8$, the analytical codebook is

$$\begin{aligned} a &= \sigma\Phi^{-1}(3/4) \simeq 0.67449\sigma, \qquad \mathcal{C}_m = \{-a, +a\}^8, \\ D_{\text{P}}(q, x) &= \|q'\|_2^2 + da^2 - 2a \sum_{j=0}^{d-1} s_j(x) q'_j. \end{aligned} \tag{5.3}$$

Φ is the standard normal CDF. Each byte stores eight signs, higher bits first. For $a > 0$, primary order equals sign-vector inner-product order. Cartesian structure also permits exact four-/eight-way splits; a learned replacement needs compatibility checks.

5.2.1 Factorisation without changing the quantiser

The high nibble selects the first four coordinates and the low nibble selects the last four: $\mathcal{C}_m[16h + l] = (A_m[h], B_m[l])$. Their tables are

$$T_m^{\text{hi}}[h] = \|q'_{m,\text{hi}} - A_m[h]\|_2^2, \qquad T_m^{\text{lo}}[l] = \|q'_{m,\text{lo}} - B_m[l]\|_2^2. \tag{5.4}$$

Disjoint squared-L2 sums give

$$T_m[z] = T_m^{\text{hi}}[z \gg 4] + T_m^{\text{lo}}[z \,\&\, 15]. \tag{5.5}$$

This stores 32 entries instead of 256, without changing real-arithmetic scores, codes or candidates.

The builder takes coordinate slices from codewords $16h$ and l. This requires all codewords sharing a nibble to share that slice. `verify_split_lut` rejects incompatible tables. An

Table 5.2. Cartesian table trade-offs for an eight-bit, eight-coordinate primary subspace. Combining additions are the algebraic minimum for joining the looked-up values, excluding accumulation across subspaces; they are not measured instruction counts.

g_{LUT}	Entries	FP32 bytes	Lookups	Combining adds	Build terms
1	256	1,024	1	0	2,048
2	32	128	2	1	128
4	16	64	4	3	32
8	16	64	8	7	16

arbitrary learned 256-centroid PQ subcodebook need not satisfy this condition: dividing its code labels into nibbles does not factor its centroid coordinates.

The same-candidate FP32 audit in section 7.3 measures numerical and ranking differences between table forms, including changes after identical primary survivor sets.

5.2.2 Grouping, construction work and resource cost

The table-grouping ablation generalises the split to $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$ contiguous coordinate groups whose bit fields match the Cartesian digits. For eight bits and $D_s = 8$, each group has $2^{8/g_{\mathrm{LUT}}}$ entries; the total entries and FP32 storage per query are

$$n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = g_{\mathrm{LUT}} 2^{8/g_{\mathrm{LUT}}}, \qquad S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = 4M n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \text{ bytes.} \tag{5.6}$$

Independent entry construction evaluates $D_s 2^{8/g_{\mathrm{LUT}}}$ coordinate contributions per subspace. Two groups therefore reduce table storage eightfold and coordinate work sixteenfold relative to independently constructing every full-table entry. These are table-level cost ratios, not predicted application speedups.

The table is built in global workspace and staged cooperatively when its complete shared-memory footprint fits the queried per-block opt-in limit $S_{\max}$. For candidate-buffer capacity C_{buf}, the ordinary exact scan and the grouped scan respectively require

$$S_0 = 8C_{\mathrm{buf}} + 8 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}), \qquad S_{\mathrm{grp}} = 8C_{\mathrm{buf}} + 24 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \quad \text{bytes.} \tag{5.7}$$

These formulas concern the evaluated scan without optional fused refinement or early-exit scratch. The code selects shared lookup exactly when the corresponding footprint is at most $S_{\max}$.

In the table sweep, $c = 1{,}000$, block sizes are 512 or 1,024, and $C_{\mathrm{buf}} = 2{,}048$. Full tables span 96–384 KiB, versus 12–48 KiB for two groups. NVIDIA specifies a 99 KiB per-block limit for compute capability 12.0 [24]. With this limit, the full-table footprint exceeds capacity even at $M = 96$ (114,696 bytes in the query-centric path), whereas the largest two-group footprint is 65,560 bytes in the grouped path. Thus the source-level placement rule sends full tables to global memory and two-group tables to shared memory in these configurations. This placement is inferred from the launch rule and published limit. The measurement records do not include a device-attribute query.

Four and eight groups both store 16 entries per subspace but perform more lookups and combinations. Extraction, lookup dependencies, staging, synchronisation and residency jointly determine the measured trade-off. Multiple scan groups may stage the same table independently.

The $16 + 16$ representation is a nibble LUT for an eight-bit sign code. Small LUTs have precedents in Fast Scan, Quicker ADC and IVF-RaBitQ [1, 2, 33]; JHQ retains its own two-level score. Our contribution is their GPU integration with retention and refinement. Two groups lead in 22/24 measured cells; other margins are below 1% (section 7.3).

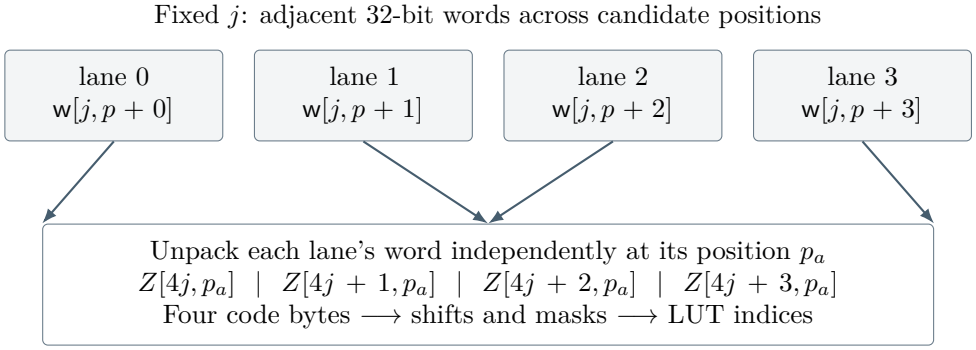


Figure 5.2. Packed candidate-parallel access, showing a subset of warp lanes. At fixed word group j, consecutive positions access adjacent four-byte words. Each word contains four subspace codes of one candidate. Unpacking preserves each code byte; LUT splitting happens afterwards. The diagram illustrates addresses, not a measured number of memory transactions.

5.3 Packed Codes and Candidate-Parallel Memory Access

Neighbouring scan lanes process neighbouring candidates. Subspace-major storage makes their code accesses adjacent; LUT factorisation separately reduces the query-specific table.

5.3.1 Address mapping and four-byte packing

For list ℓ, row r has position $p = o_\ell + r$. Code $Z[m, p]$ lies at byte offset $mN + p$, rather than candidate-major $pM + m$. Adjacent lanes then read adjacent candidates at fixed subspace m. A warp crossing lists can still jump in memory.

Four subspace bytes from one candidate form one word:

$$\mathsf{w}[j, p] = \sum_{t=0}^{3} 2^{8t} Z[4j + t, p], \qquad \text{offset}_{\text{bytes}}(\mathsf{w}[j, p]) = 4(jN + p). \tag{5.8}$$

Shifts and masks recover each byte exactly: $(\mathsf{w}[j, p] \gg (8t))$ & 255. Its nibbles index equation (5.5). This packs subspaces of one vector, not four candidates.

The build path explicitly repacks the $[M, N]$ byte array into $[M/4, N]$ words. Merely reinterpreting the original byte array as `uint32_t` would combine consecutive candidate bytes rather than the required subspaces. The implementation requires M divisible by four and rejects a partial final word. The repack reads and writes NM code bytes once at construction, with old and new storage briefly coexisting; it is outside the search timing boundary.

5.3.2 What packed access saves, and what it preserves

One word load replaces four byte loads, followed by register unpacking. For 32 positions the logical window is 128 bytes; alignment and active lanes determine transactions. Payload remains M bytes per candidate. Since both alternatives are subspace-major, the experiment isolates neither transpose nor cache effects.

Section 7.3 evaluates the packed path's throughput, recall and variability against the byte path.

5.4 Exact Global Primary Top-c via Multi-Block Scanning

Grouped scanning adds blocks within each query, then merges survivors [18]. It preserves computed-primary selection, not exact raw-vector neighbours.

5.4.1 Candidate partition and deterministic ordering

For a fixed query, concatenate the selected inverted lists in probe order to obtain n_c candidate occurrences. Assume that each visited database row occurs once and that the index

stays fixed during search. For $g_{\text{scan}} \in \{1, 2, 4, 8\}$, block (q, j) owns the interval

$$U_j = \left[\left\lfloor \frac{jn_c}{g_{\text{scan}}} \right\rfloor, \left\lfloor \frac{(j+1)n_c}{g_{\text{scan}}} \right\rfloor \right), \qquad j = 0, \ldots, g_{\text{scan}} - 1. \tag{5.9}$$

Here an interval denotes the corresponding candidates, rather than their database IDs. The device computes its endpoints from the routed candidate count. Groups differ in size by at most one candidate; a long list can cross a group boundary. This balances candidate counts, whereas assigning equal numbers of lists need not do so. Each thread finds its first list through the probe-prefix offsets and advances its list cursor while scanning its interval. Residual codes are not read at this stage.

The selection key is

$$K_q(x) = \big(D_{\text{P}}(q, x), p(x)\big), \qquad (a, p) \prec (b, r) \iff a < b \text{ or } (a = b \text{ and } p < r), \tag{5.10}$$

where $p(x)$ is the physical row position in the fixed inverted-list storage. It is unique among the query's candidates and independent of the order of atomic appends. It is not the database ID used to break final-result ties. With non-NaN scores, this defines a total order on distinct candidates. All local comparisons and merge decisions use that same order. Partition-independent selection additionally requires that each candidate receive the same computed score in every compared path; the ordering rule alone cannot remove floating-point scoring differences.

5.4.2 Exact local retention and global coverage

$\text{Top}_c(V)$ is the first $\min(c, |V|)$ elements under K_q, with $c = \max\{k, \lceil \alpha k \rceil\}$. Each group needs its full local top-c: all global winners could lie in one group. A c/g_{scan} quota can lose them.

Lemma 5.1 (Local-buffer retention)**.**
Let V_t be the candidates scored by a block through chunk t, and B_t its valid buffer after any compaction or filtering at that point. Assume fixed complete keys, sufficient append capacity and exact top-c compaction. Let β_t be the minimum of the complete shared bounds observed so far, including the initial top sentinel. Then

$$\text{Top}_c(V_t) \cap \{x : K_q(x) \preceq \beta_t\} \subseteq B_t. \tag{5.11}$$

With sharing disabled, the bound is the sentinel and the buffer contains the whole prefix top-c.

Proof.
The empty prefix satisfies the invariant. Local rejection has c earlier witnesses, so the rejected key cannot become a later prefix winner. Shared rejection excludes only keys above β_t. Every other eligible winner is already retained or appended. Exact compaction keeps it because fewer than c prefix keys precede it; filtering removes only keys above an observed bound. Induction over these operations proves the claim. The running minimum handles stale reads of previously published bounds. □

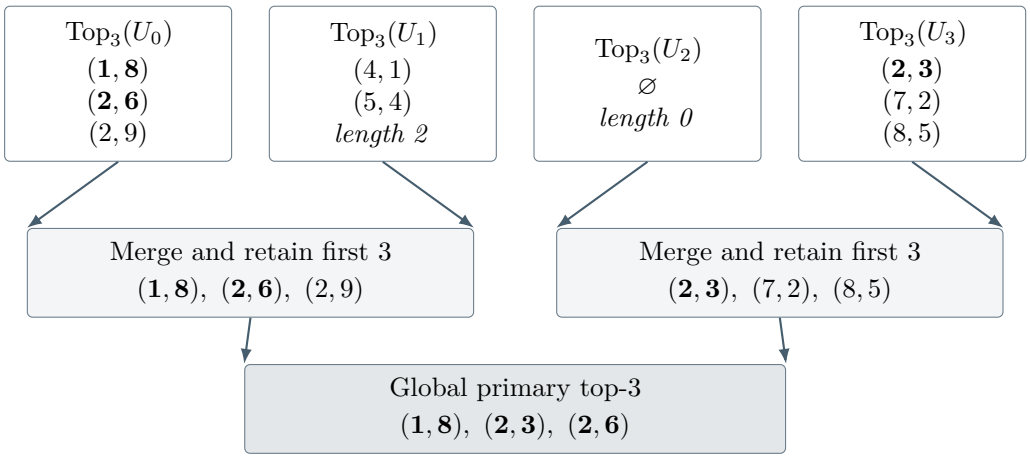


Figure 5.3. Local coverage and truncated pairwise merging, illustrated with $c = 3$ and $g_{\text{scan}} = 4$. Entries are (D_{P}, p) keys, not measured data. Bold entries are the final global winners. A short group contributes its valid prefix; an empty group contributes no entries. Every merge uses the same key order and retains at most c entries.

Proposition 5.1 (Coverage and truncated merging)**.**
For fixed keys under a common total order, let U be the visited universe and V any retained subset. If $\text{Top}_c(U) \subseteq V \subseteq U$, then $\text{Top}_c(V) = \text{Top}_c(U)$. Pairwise sorted merging with truncation to c preserves this result. In particular, for disjoint groups,

$$\text{Top}_c\left(\bigcup_j U_j\right) = \text{Top}_c\left(\bigcup_j \text{Top}_c(U_j)\right). \tag{5.12}$$

The statement includes universes smaller than c.

Proof.
Every global winner is in V, and every other member of V follows the global winners. Selection from V therefore recovers them. For the independent-group special case, let x belong to the global top-c and to group U_j. Fewer than c candidates precede x globally, so fewer than c precede it within U_j. Consequently $x \in \text{Top}_c(U_j)$, and the local union contains every global winner. Selecting its first c therefore recovers precisely the global winners in the same order. If the whole universe has fewer than c members, every member is retained.

For merging, the first c of two sorted lists are their union's top-c. A discarded candidate already has at least c predecessors within that union and cannot enter the top-c of any larger union. Induction over the merge tree establishes the claim. □

The CUDA implementation maintains a shared-memory buffer of C_{buf} key pairs, where C_{buf} is a power of two and $C_{\text{buf}} \geq c + T_{\text{blk}}$ for a scan block of T_{blk} threads. A compaction sorts this buffer with a bitonic network, retains at most c valid entries and records the last retained key as the local threshold when c entries exist. Before that point the threshold is a sentinel above every valid key. Between compactions, only candidates preceding the threshold are appended. Its c witness candidates make the threshold a safe upper bound for the best c of the scanned prefix, even as further candidates arrive.

Each chunk appends at most T_{blk} entries. Compaction precedes any chunk that would exceed capacity, leaving at most c entries and space for the next chunk because $C_{\text{buf}} \geq c + T_{\text{blk}}$. This supplies the capacity premise of Lemma 5.1 without a per-thread quota. With sharing, Proposition 5.2 bounds every observed threshold above the global cth key. The buffer lemma and coverage proposition therefore preserve global winners through filtering and merging.

Materialising all n_c score/position pairs needs $O(n_c)$ writes/storage. Fused scoring instead repeatedly sorts bounded local buffers and emits $O(g_{\text{scan}}c)$ entries. Register selection is another established option [18]; the experiments test the chosen scan variants.

5.4.3 Merge implementation, storage and boundary cases

For $g_{\text{scan}} > 1$, one 256-thread block merges each query over $\log_2 g_{\text{scan}}$ rounds. Each pair emits $\min(c, a + b)$ entries. Merge Path finds thread slices by binary search, then each thread merges its slice [14]. All comparisons use equation (5.10); block barriers separate rounds. Figure 5.3 illustrates the truncation invariant. The tree emits at most $c(g_{\text{scan}} - 1)$ key pairs across its internal nodes, in addition to the partition searches and synchronisation work.

The two global score/position workspaces have capacities $[b_{\text{cap}}, g_{\text{scan}}, c]$ and $[b_{\text{cap}}, g_{\text{scan}}/2, c]$ and alternate between rounds. With four-byte scores and positions, their combined payload is

$$S_{\text{pairs}} = 8b_{\text{cap}}c\left(g_{\text{scan}} + \frac{g_{\text{scan}}}{2}\right) \quad \text{bytes}, \qquad g_{\text{scan}} > 1. \tag{5.13}$$

Group counts require another $4b_{\text{cap}}g_{\text{scan}}$ bytes; shared bounds add $8b_{\text{cap}}$ when provisioned. These are additional to final survivor arrays and other search workspace. Allocation grows before graph capture and is retained for reuse, so choosing a smaller g_{scan} later does not necessarily release the reserved memory. The merge keeps only list lengths in shared memory rather than all $g_{\text{scan}}c$ keys.

Valid counts delimit each local prefix. Sort padding uses $(+\infty, \text{INT_MAX})$; valid positions must be smaller. Empty groups have count zero. Output padding instead uses $(+\infty, -1)$, which refinement skips and merging must not treat as valid. $g_{\text{scan}} = 1$ writes survivors directly, without merging.

$g_{\text{scan}} = 0$ labels one block per query. Its default selector is exact by distance but can choose different cutoff-tie IDs from the complete-key path. The lossy `K_LOCAL=4` alternative requires `JHQ_EXACT_TOPCK=0`; paper-faithful builds disable it.

For distance-only compaction, every rejected score has c retained witnesses no larger than it. The final buffer therefore contains all scores below the cutoff and enough equal scores, without fixing tied IDs. T4 checks this preorder and complete-key grouped selection (section 7.4.1). The reference sorts GPU-produced scores, rather than recomputing original-space distances on the CPU.

5.5 Shared Complete-Key Thresholds and Execution Scheduling

Full local top-c retention establishes correctness, but can produce $g_{\text{scan}}c$ intermediate survivors for only c global outputs. The shared-bound component reduces this excess selection work. The scheduler then chooses how much scan parallelism to expose. Both operate at a fixed refinement budget; choosing that budget is the separate problem in Chapter 6.

5.5.1 A complete-key upper bound with explicit witnesses

Let $K_{(c)}$ be the global cth key when $|U| \geq c$. If a group holds c distinct, fully scored candidates, the largest key among those witnesses, τ_j, satisfies $K_{(c)} \preceq \tau_j$. There are already c candidates no later than τ_j in the global universe. The shared bound is initialised to a top sentinel and updated by

$$\tau \leftarrow \min_{\prec}(\tau, \tau_j). \tag{5.14}$$

No bound is published from fewer than c valid candidates. If the entire universe has fewer than c, no group can publish such a bound and all valid candidates remain eligible.

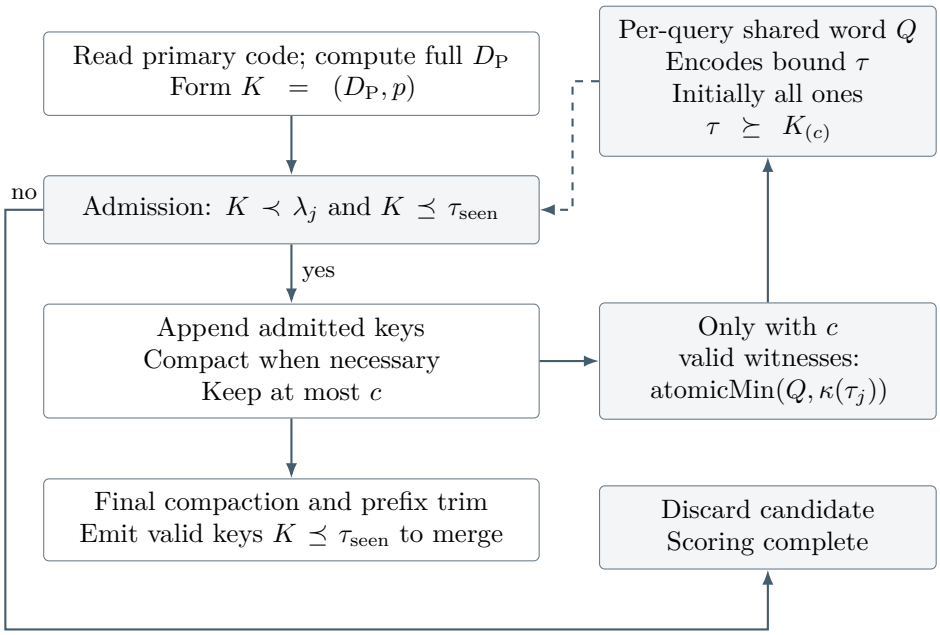


Figure 5.4. Shared complete-key filtering for one scan group. A candidate's full primary score is computed before admission. A valid c-candidate witness set can tighten the per-query bound; a stale but complete bound only weakens filtering. Final prefix trimming reduces the valid merge input, while the allocated workspace capacity remains unchanged. The whole-key-read premise is qualified in the text.

Proposition 5.2 (Safe rejection by a witnessed bound)**.**
Assume fixed keys, publications backed by c distinct candidates, and reads that return either the initial sentinel or one complete published key. Rejecting a candidate only when $K_q(x) \succ \tau$ cannot remove a global top-c candidate, irrespective of publication timing.

Proof.
Every published bound is at least $K_{(c)}$, and the minimum of such bounds is also at least $K_{(c)}$. A global winner satisfies $K_q(x) \preceq K_{(c)} \preceq \tau$, so it is never rejected. An older bound remains an upper bound and merely permits extra work. Publications made after earlier safe rejections remain valid: any c retained distinct candidates still supply c witnesses, whether or not they are the best c of their group. □

Sharing may remove local winners certified as globally uncompetitive; the retained union still satisfies Proposition 5.1. Equality with the shared bound remains eligible. Admission requires

$$K_q(x) \prec \lambda_j \quad \text{and} \quad K_q(x) \preceq \tau_{\text{seen}}, \tag{5.15}$$

where λ_j is the local compaction threshold and τ_{seen} is the group's last observed shared bound.

5.5.2 Publication, filtering and implementation limits

Nonnegative FP32 squared-sum scores and positions are packed as

$$\kappa(D, p) = 2^{32}\,\text{uintbits}_{32}(D) + p. \tag{5.16}$$

Unsigned comparison matches lexicographic order for positive-zero, non-NaN scores and valid nonnegative positions. Infinity follows finite scores; the initial all-one word follows valid keys. Invalid positions cannot publish bounds.

One thread polls the bound per chunk. After compaction, c valid entries permit publishing the local cth key with 64-bit `atomicMin`. A final publication and binary search trim the sorted

prefix. Groups never wait for one another: stale bounds leave extra entries for exact merging. Each search resets bounds in-stream; merging follows scan completion.

Extracted PTX uses aligned `ld.volatile.global.u64` and `atom.global.min.u64`. PTX's single-copy atomicity for compatible same-size strong accesses supplies the whole-key premise [26]. CUDA 13.0 `ptxas` for `sm_120` corroborates this (section A.4); actual driver-JIT SASS was not captured. The argument does not rely on C++ `volatile` alone [25].

Bounds are checked after full primary scoring (figure 5.4). They save buffer/merge work, not distance arithmetic, and add polling/atomics. Section 7.4 includes a slowdown.

5.5.3 Choosing parallelism without changing the budget

Grouping launches bg_{scan} blocks rather than b, adding parallelism but duplicating staging/selection and requiring merging. Smaller static quotas would reduce intermediate storage, but require a certificate or fallback to preserve coverage; the pilot's concentrated survivor distribution motivated using witnessed thresholds instead (Section 7.4).

The implemented scheduler uses a fill proxy and a workload-skew statistic. Let T_{blk} be scan threads per block, H the number of SMs times the maximum resident threads per SM, and $v = \text{sd}(n_c)/\,\text{mean}(n_c)$ the candidate-count coefficient of variation. Its rule is

$$g_{\text{fill}} = \max(\{1\} \cup \{h \in \{2,4,8\} : bhT_{\text{blk}} \le 0.6H\}), \tag{5.17}$$

$$g_{\text{skew}} = \begin{cases} 4, & v \ge 0.3, \\ 1, & v < 0.3, \end{cases} \qquad g_{\text{chosen}} = \max(g_{\text{fill}}, g_{\text{skew}}). \tag{5.18}$$

The skew rule may override the fill limit. In the evaluation harness, v is calibrated using 512 database vectors as pseudoqueries and a fixed routing configuration; it uses no ground-truth labels. This is distinct from Chapter 6's sampled output-agreement calibration. Once calibrated, scheduling uses the batch size and stored statistic without an additional per-search host readback. The constants were fitted on four pilot configurations and then evaluated on further configurations. The one-time pseudoquery calibration is excluded from the reported steady-state search times. Supplementary timing separates host gathering (0.81–7.04 ms), the first calibration call (6.00–27.77 ms) and repeat medians (2.72–18.13 ms). This startup cost is separate from refinement-budget calibration (section A.4).

A chosen group count of one dispatches $g_{\text{scan}} = 0$, keeping index, probes and budget fixed but inheriting distance-only cutoff ties. Identical survivors thus require a common-order check. Integrated checks, gains and regret appear in section 7.6.

5.5.4 Observed execution paths

On the 170-SM RTX 5090, 48 warps/SM gives $0.6H = 156{,}672$ threads [24]. At batch 1,000 and $T_{\text{blk}} = 512$, two groups need a 1,024,000-thread proxy. Fill therefore chooses one; skew can force four.

Nine of 38 E1 run records cross $v = 0.3$, all Vogue (0.338–0.622). Their 132 run–budget cells contain 27 Vogue cells scheduled to four groups and 105 arXiv/OpenAI cells scheduled to the query-centric path. Non-Vogue full-batch gains therefore use LUTs, packed access, lane refinement and probe selection, without grouped retention or shared bounds.

Vogue checks find 3–15 tie-only primary differences per 1,000 queries, zero non-tie differences and identical final outputs. The 7–11 range applies only at 128 probes.

Held-out budget tests use one block in ten cells and four groups in two Vogue cells. At $S = 128$, eight use two groups, two use four and two use one block: 8/12 change paths. Grouping mainly serves smaller/skewed workloads, without a fixed batch-256 cutoff.

The heuristic omits register/shared-memory residency, compaction and merge costs, and its pseudoqueries can become stale. Its worst recorded regret is 1.217 at batch 256. Measurements hold the refinement budget fixed; section 5.6 processes the resulting survivors.

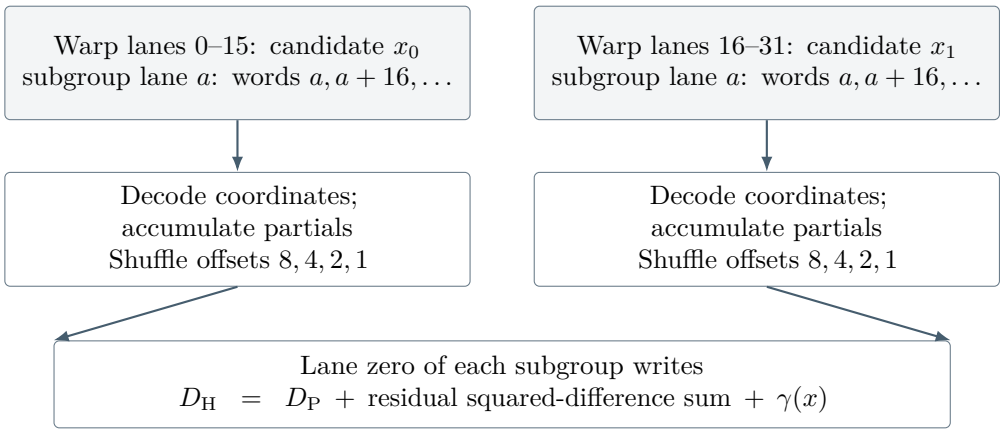


Figure 5.5. Sixteen-lane residual refinement. A warp handles two candidates; lane a within a candidate reads residual words $a, a+16, \ldots$. With eight-bit residual codes, each 32-bit word decodes four coordinates. Subgroup reductions combine partial scores before adding the retained primary score and correction. Candidate positions can be scattered even when each subgroup's word access is contiguous.

5.6 Lane-Cooperative Residual Scoring and Final Top-k

Residual access is restricted to primary survivors, reducing coordinate work from $O(n_c d)$ to $O(cd)$. Lane cooperation accelerates that work at a fixed budget.

5.6.1 Reconstructed scoring and the query-only constant

With primary reconstruction $\widehat{p}$, residual $\widehat{r}$ and stored correction $\gamma = 2\langle \widehat{p}, \widehat{r} \rangle$, refinement computes

$$D_{\mathrm{H}}(q, x) = D_{\mathrm{P}}(q, x) + \|q' - \widehat{r}(x)\|_2^2 + \gamma(x) \tag{5.19}$$

$$= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2. \tag{5.20}$$

Norm expansion gives the second equality. The query-only constant preserves ranking, while reconstruction error remains [16].

The scalar path sums coordinate-wise residual differences from q', staging queries and, when possible, codebooks in shared memory. It then adds the primary score and correction.

5.6.2 Mapping lanes to words and reducing partial sums

At $\ell_{\mathrm{ref}} = L \in \{1, 8, 16, 32\}$, adjacent lanes share one candidate. A residual word holds $d_{\mathrm{word}} = 32/B_R$ codes: four for eight-bit, eight for four-bit. Lane a reads words $a, a+L, \ldots$; butterfly shuffles sum partials and lane zero writes the score.

Rows must be word-aligned and $d_{\mathrm{word}} \mid D_s$, keeping words within a codebook. All lanes join shuffles; invalid slots contribute zero, then receive infinite output scores. At $L = 16$, each warp handles two candidates.

L0 uses scalar byte loads; L1 packs word loads but keeps one lane and the accumulation order. L8/L16/L32 divide coordinate work, shortening each walk at the cost of shuffles and fewer concurrent candidates. Larger L need not win; survivor locality and codebook placement still matter.

5.6.3 Final selection and the numerical boundary

Given k finite valid survivors with unique IDs, one block selects exact top-k under $(D_{\mathrm{H}}, \mathrm{ID})$. Each round scans all c slots, emits the minimum and sets its score to infinity. There is no per-thread quota. If finite entries run out, a valid smallest ID can repeat with infinity; an entirely invalid pool returns invalid IDs. Evaluated pools satisfy the finite-entry contract.

Final ties use database IDs; primary ties use positions. Refinement cannot restore excluded candidates or remove quantisation error. Lane reductions change FP32 association and may move near ties.

Section 7.3 reports the load-only and cooperative controls, numerical differences, kernel times and end-to-end gains separately.

5.7 IVF Probe Selection as an Independent Primitive

Probe selection precedes primary scanning and selects inverted lists, not database-vector survivors. For IVF centroid μ_i, the selector orders the FP32-valued computed routing scores

$$s_i = \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, \qquad (s_i, i) \text{ ordered lexicographically.} \tag{5.21}$$

Omitting the common query norm preserves centroid ordering. In contrast to primary squared-distance tables, these scores can be negative. The required output is the first $n_{\text{sel}} = \min(n_{\text{probe}}, n_{\text{list}})$ centroid IDs in score order, using the smaller list ID at a tie, plus the prefix offsets of their list lengths.

OpenAI rotation and centroid GEMMs use TF32 inputs with FP32 accumulation in v69. Probe selection is exact for those computed scores; later FP32 `fma` cannot undo GEMM rounding (section 7.2.3).

5.7.1 Radix selection followed by deterministic ordering

The radix replacement uses one block per query and four most-significant-byte-first passes. Each histogram includes the chosen prefix only. Selecting its next bin updates the remaining rank; four passes determine the cutoff and required equal-key count.

Signed zeros first map to positive zero; then float bits u map to

$$F(u) = \begin{cases} \sim u, & \text{if the sign bit is set,} \\ u \mid 2^{31}, & \text{otherwise.} \end{cases} \tag{5.22}$$

This orders non-NaN values, including negatives and infinities, unlike equation (5.16). Equal zeros share a key. Every pass uses the same rounded `fma`, keeping recomputed keys fixed.

The 256-bin histogram has 32 lane replicas: 32 KiB of counters. This separates same-warp updates but leaves inter-warp contention, trading shared storage and reduction for less concentrated atomics.

Collection takes all keys below cutoff. Ballots and prefix counts rescan cutoff ties in list-ID order, so atomic arrival cannot choose the tied subset. Each selected entry then counts its predecessors under (s_i, i) and writes to that rank, avoiding a full centroid sort.

A block prefix scan converts selected list lengths into offsets. Excess probes repeat the last ID with zero length, so they do not rescan its list. Centroid sets and requested probe count must be nonempty/positive.

5.7.2 Costs and separate correctness contracts

Histogram/collection passes reread scores; ranking costs $O(n_{\text{sel}}^2)$. Larger selections also need more shared storage, while small ones still pay histogram allocation. We evaluate this existing radix method's deterministic integration, not a uniformly linear replacement.

Tests include cutoff ties, signed zeros, infinities, extreme values and excess probes (section 7.4).

The optional FP16 routing changes score precision and is evaluated as a separate approximation in section 7.4, alongside the cross-process pipeline-identity checks.